



СПбГЭТУ «ЛЭТИ»
ПЕРВЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ

ВСТРАИВАЕМЫЕ СИСТЕМЫ

4 курс

Авторы: Голик С.Е., Путов А.В., Игнатьев К.В., Копычев М.М.

Преподаватели: Копычев Михаил Михайлович
Кузьмина Татьяна Олеговна

(Аудиторные работы)

Санкт-Петербург 2026

1. Описание лабораторного стенда и инструментальных средств разработки прикладных программ

1.1. Описание лабораторного стенда

Лабораторный стенд представляет собой аппаратно-программный комплекс с совокупностью объектов управления, датчиков и средств индикации, управляемых микроконтроллером (МК) Atmega128A, работающем на частоте 11059200 Гц или 11,0592 МГц. На рис. 1.1 приведена схема передней панели лабораторного стенда.

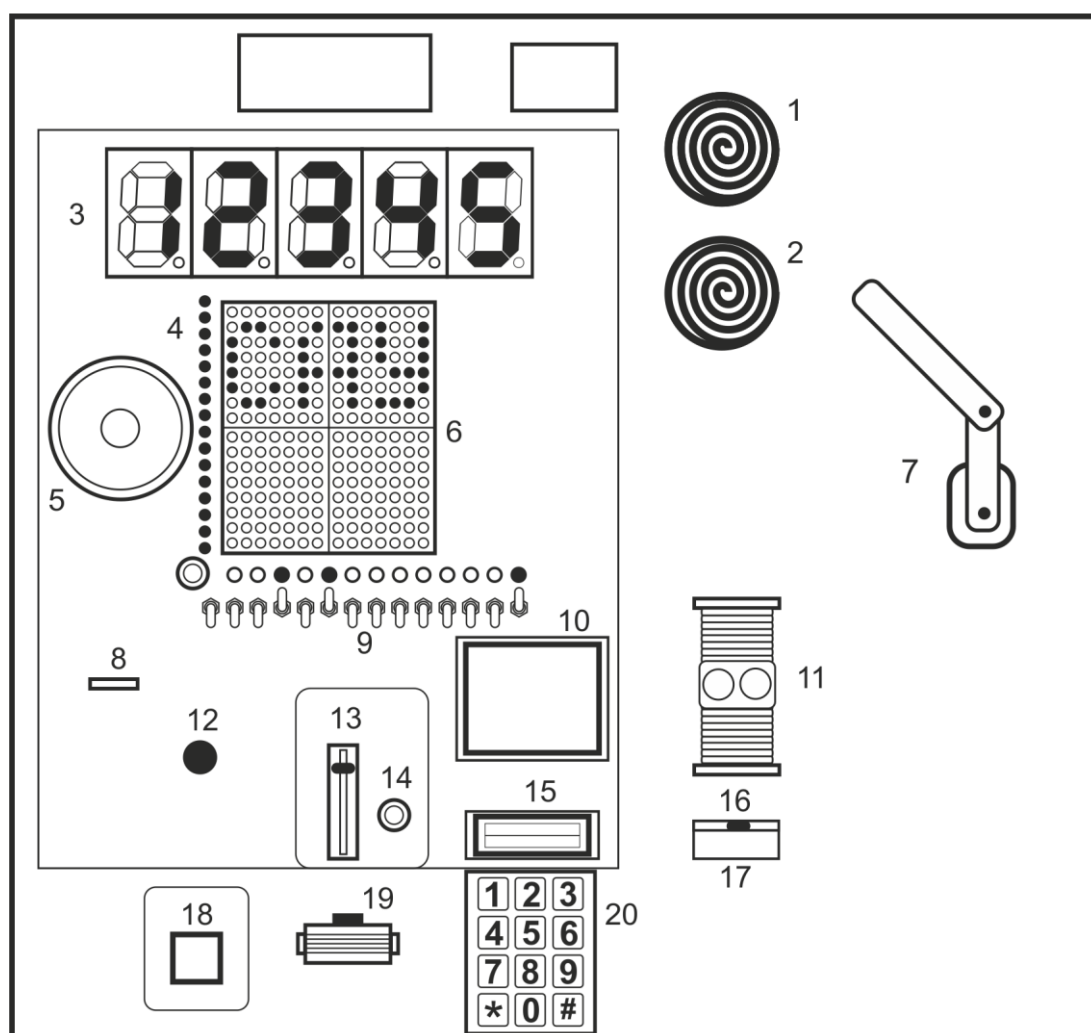


Рис. 1.1. Лабораторный стенд

В состав лабораторного стенда включены следующие компоненты:

- шаговый двигатель с силовым драйвером на базе микросхемы транзисторных ключей ULN2003A;
- двигатель постоянного тока с силовым широтно-импульсным преобразователем на микросхеме L298N;

- пятиразрядный цифровой дисплей на семисегментных индикаторах со схемой управления;
- светодиодная шкала из 16 светодиодов, разделенная на две автономные части по 8 светодиодов в каждой;
- динамический громкоговоритель, предназначенный для формирования звуковых сигналов;
- светодиодная матрица 16x16 точек, состоящая из четырех квадратных матриц размера 8x8;
- мини манипулятор «серворука», состоящая из двух суставов с сервомашинкой в каждом суставе;
- слот для SD карты памяти;
- тумблеры конфигурации стенда;
- графический монохромный жидкокристаллический индикатор размером 128x64 точки WG12864A;
- ультразвуковой дальномер;
- многоцветный светодиод RGB (Red, Blue, Green) для формирования световых сигналов разного цвета;
- ползунковый потенциометр, используемый в качестве датчика;
- инкрементный энкодер, используемый в качестве датчика угла поворота вала;
- символьный монохромный жидкокристаллический индикатор размером 2 строки по 16 символов точки WH1602C;
- магнитный подвес с катушкой, формирующей силовое магнитное поле;
- датчик Холла магнитного подвеса
- тензометрический датчик для измерения момента;
- цифровой температурный датчик DS1631S в связке с мощным резистором, используемым для изменения температуры;
- мембранная клавиатура 4x3 клавиши.

Большая часть перечисленных компонентов стенда используются в описываемых ниже лабораторных работах в качестве подключенных к МК Atmega128A объектов управления, датчиков и средств индикации.

1.2. Введение в интегрированную среду разработки *Atmel (Microchip) STUDIO*

МК может быть запрограммирован с компьютера со средой разработки и с помощью устройства, называемого программатором. Среда разработки предназначена для написания кода, компиляции и перевода языка C (например) в двоичный код, который может быть понят МК.

Среда разработки может использоваться одна из множества. Воспользуемся Atmel STUDIO (далее – STUDIO) (см. рис. 1.2).



Рис. 1.2. Atmel STUDIO (STUDIO)

Atmel STUDIO – основанная на Visual STUDIO бесплатная проприетарная интегрированная среда разработки (IDE) для разработки приложений для 8- и 32-битных микроконтроллеров семейства AVR и 32-битных микроконтроллеров семейства ARM от компании Atmel. STUDIO содержит компилятор GNU C/C++ и эмулятор, позволяющий отладить выполнение программы без загрузки в микроконтроллер.

На первом этапе исходный код программы пишется на ассемблере или языке C. Этот этап выполняется с помощью текстового редактора, который является частью IDE, предназначенного для создания и изменения текста исходных программ и записи их в файл.

Перевод программы с C в машинные коды выполняется с помощью программного модуля транслятора. Переводчик не только переводит языковые команды в машинные инструкции, но и обнаруживает синтаксические ошибки в исходной программе с указанием их местоположения.

Следующий шаг – компиляция/линковка – несколько автономно транслируемых программных модулей объединяются (компилируются) в единый модуль для загрузки его в МК. Необходимость этой операции

объясняется наличием ссылок в программных модулях. Эти ссылки относятся к другим программным модулям и модулям заголовочных файлов. Кроме того, в соответствии с информацией, помещенной транслятором в программный модуль, компоновщик перемещает адреса этого модуля в соответствии с фактическим расположением программы в программной памяти МК. Компоновщик отслеживает ошибки сборки программного модуля и предоставляет информацию об этих ошибках.

STUDIO объединяет все описанные выше шаги разработки прикладной программы в единый рекурсивный процесс, в котором в любой момент вы можете быстро вернуться к любому предыдущему этапу, чтобы исправить ошибки или изменить алгоритм.

На рис. 1.3 показан пользовательский интерфейс STUDIO .

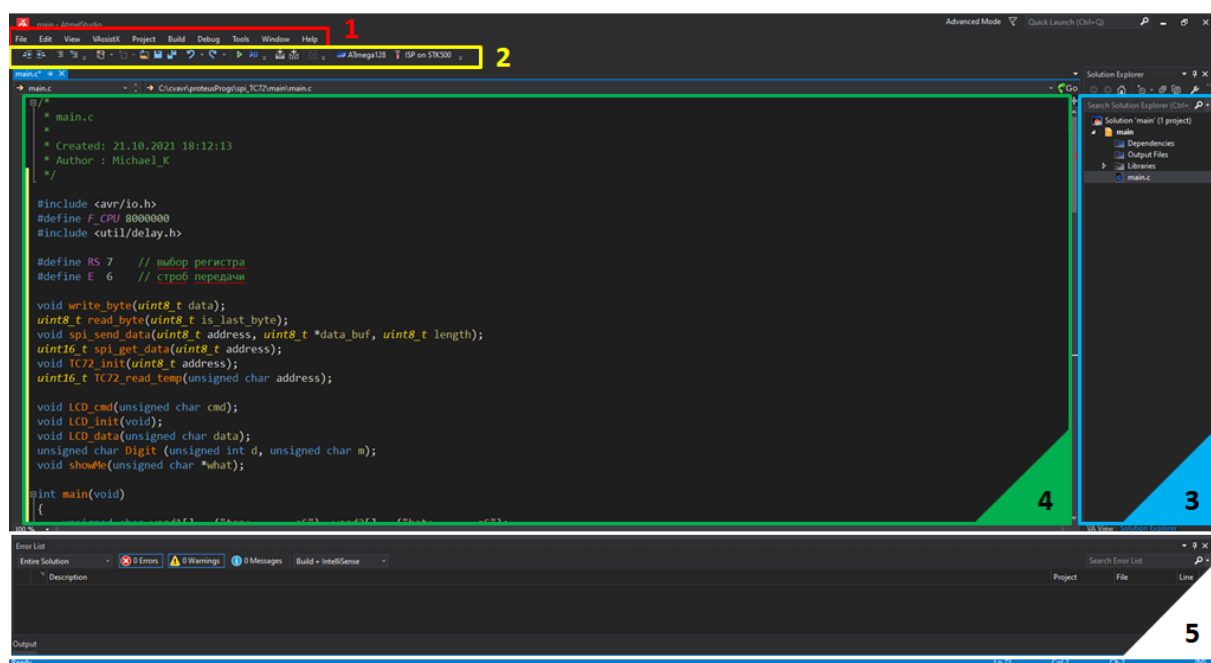


Рис. 1.3. Пользовательский интерфейс STUDIO

На рис. 1.3 цифра 1 – строка меню, 2 – кнопки управления (настраиваемые), 3 – структура текущего проекта (solution), 4 – редактор текста, 5 – информационное окно.

Для написания новой программы, необходимо создать новый проект. Для этого перейдите **File** → **New** → **Project** (см. рис. 1.4).

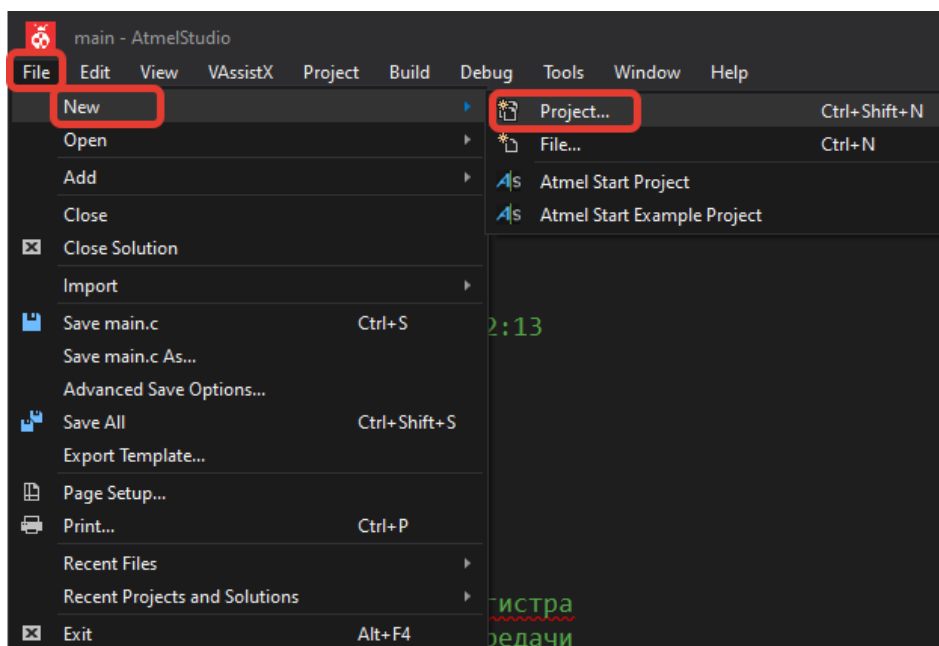


Рис. 1.4. Новый проект

У нас откроется диалог, в котором будет несколько вариантов проектов на C/C++. Мы будем писать именно на языке C, выбираем **GCC C Executable Project** (см. рис. 1.5).

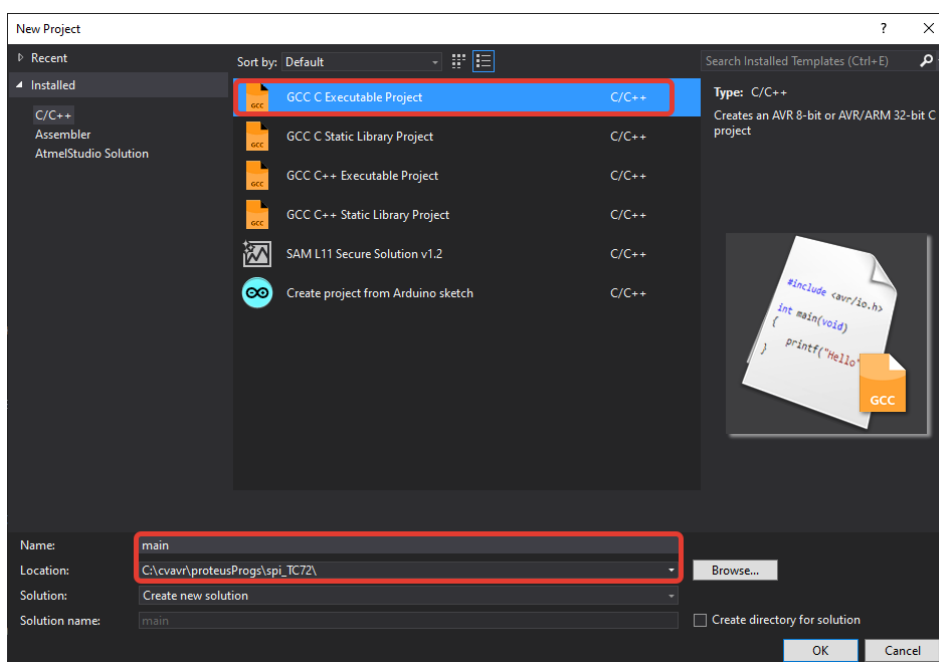


Рис. 1.5. Выбор типа проекта

В поле **Name** (имя) задаётся название файла проекта, в поле **Location** (место сохранения) выберите диск и папку, куда вы хотите сохранить свой проект.

РЕКОМЕНДАЦИЯ: создавайте новую папку для каждого вашего нового проекта.

После этого создаётся новый файл проекта в указанном месте на жестком диске компьютера. См. рис. 1.6 и 1.7.

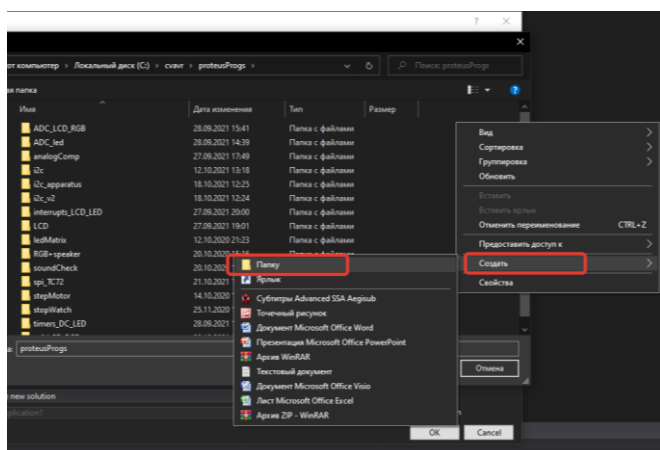


Рис. 1.6. Создавайте новую папку для каждого проекта

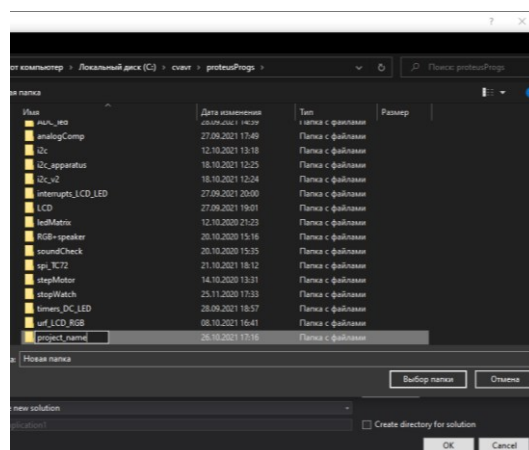


Рис. 1.7. Название проекта не должно содержать специальных символов

После создания папки, присвоения имени проекту и нажатия кнопки «Сохранить» появляется следующее окно (см. рис. 1.8). Теперь необходимо выбрать МК, с которым мы будем работать.

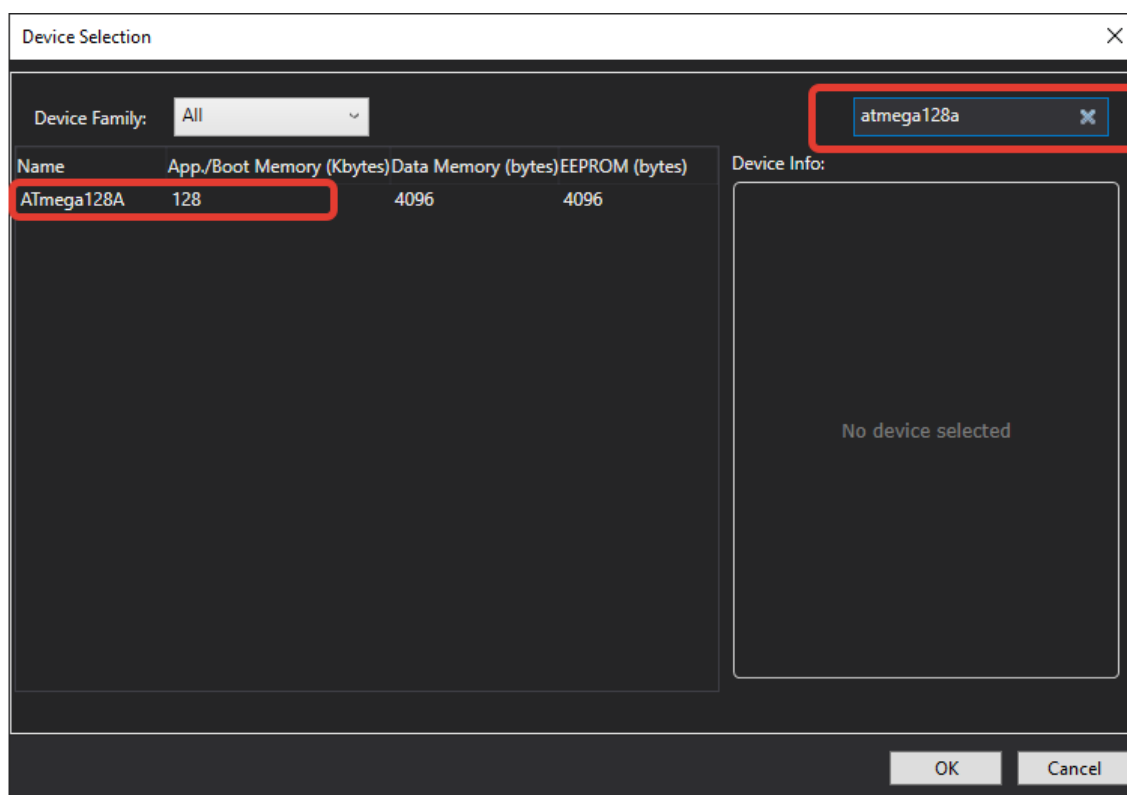


Рис. 1.8. Окно выбора микроконтроллера

Мы работаем с Atmega128A, поэтому выбираем её. После этого появится похожее окно (см. рис. 1.9), в котором мы и будем писать нашу программу, отлаживать её и записывать на контроллер.

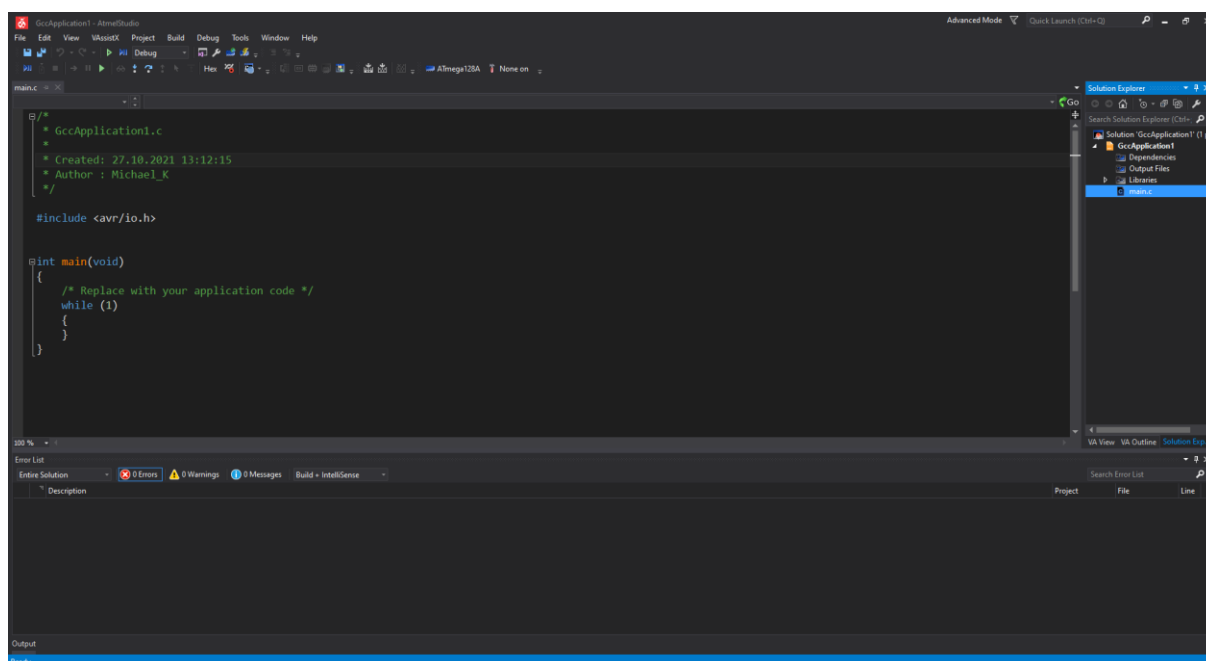


Рис. 1.9. Рабочий экран STUDIO

Немного настроим наш проект. Перейдите по вкладкам Project → *project_name* Properties (см. рис. 1.10).

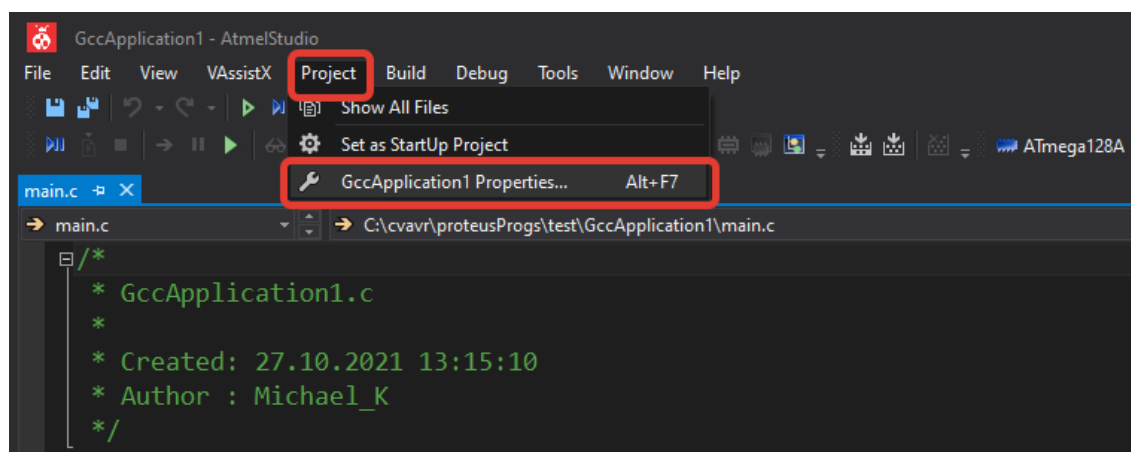


Рис. 1.10. Перейдем к свойствам проекта

В открывшемся диалоговом окне (см. рис. 1.11) перейдем на вкладку *Tool* и выберем свой программатор.

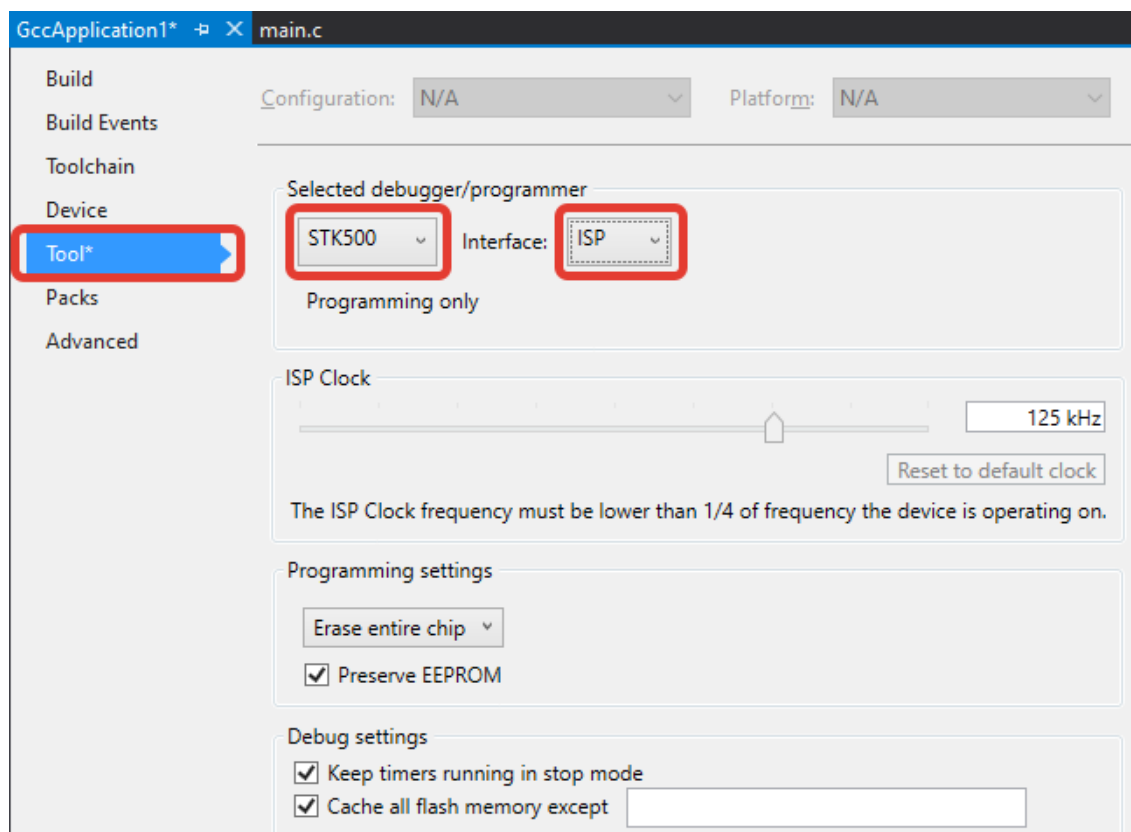


Рис. 1.11. Свойства проекта

После того, как код написан, его нужно собрать. Для сборки кода перейдите в **Project** → **Build Solution** (блок 1 либо пиктограмма в блоке 2 на рис. 1.3) или нажмите F7. Если в информационном поле проекта нет ошибок (в том числе предупреждений) (блок 5 на рис. 1.3) и пользователь считает, что код работает, можно загрузить программу в МК.

¹Если у пользователя есть аппаратный МК и подключенный к нему программатор, запустите **Device Programmer Tools** → **Device Programmer** (блок 1 на рис. 1.3) или нажмите комбинацию Ctrl + Shift + P. Появится новое окно. В этом окне необходимо настроить поля **Tool** (виртуальный COM порт программатора), **Device** (микроконтроллер) и **Interface** (программатор). После этого можно попробовать нажать **Read** и убедиться, что связь компьютера с микроконтроллером через программатор налажена (мы прочитаем подпись контроллера). Чтобы записать собранный код на микроконтроллер, следует перейти на вкладку **Memories**, после чего очистить микроконтроллер нажатием кнопки **Erase now**, после этого можно записать код на микроконтроллер нажатием кнопки **Program** в области **Flash**. Все описанные в абзаце действия

¹ При работе с Proteus данный абзац можно не читать

можно заменить нажатием кнопки **Start without debugging** (Ctrl + Alt + F5) в меню **Debug** (блок 1 на рис. 1.3).

Для адресации отдельных битов операндов удобно использовать битовые операции, такие, как *И* (&), *ИЛИ* (|), *исключающее ИЛИ* XOR (^), отрицание (~) и сдвиги вправо (>>) и влево (<<). *И* является аналогом умножения, *ИЛИ* является аналогом сложения, *XOR* является аналогом сложения по модулю два, отрицание является аналогом инверсии.

Например, требуется настроить вывод 4 порта *D* как выход:

```
DDRD = (1<<4); //DDRD = 0b00010000
```

Сдвиговая операция с 1 (0b00000001) помогает поместить логическую единицу на любое место (0...7). Таким образом, $(1<<n)$ эквивалентно помещению логической единицы в бит № *n*, а в остальные биты помещаются логические нули. Пусть необходимо настроить вывод 6 порта *D* как выход:

```
DDRD = (1<<4); //DDRD = 0b00010000
```

```
DDRD = (1<<6); //DDRD = 0b01000000
```

Если необходимо сохранить состояние 4^{ого} бита в дополнение к 6^{ому}:

```
DDRD = (1<<4); //DDRD = 0b00010000
```

```
DDRD |= (1<<6); //DDRD = 0b01010000
```

//или DDRD = DDRD | (1<<6) добавление нового DDRD к старому

Операция | помогает сохранить предыдущее состояние операнда и, кроме того, установить один или несколько битов в единицу.

Теперь требуется настроить 4^{ый} вывод порта *D* как вход, но сохранить состояние 6^{ого} вывода:

```
DDRD = (1<<4); //DDRD = 0b00010000
```

```
DDRD |= (1<<6); //DDRD = 0b01010000
```

```
DDRD &= ~(1<<4); //DDRD = 0b01000000
```

//или DDRD = DDRD & ~(1<<4)

Таким образом, операция &= ~ позволяет сохранить предыдущее состояние операнда и, кроме того, установить один или несколько битов в ноль.

2. Быстрый старт

Методика Quick Start (Быстрый старт), введенная в обиход производителями современных средств разработки программ, ставит своей целью показать на простом примере основные шаги по использованию предлагаемой IDE. Данная лабораторная работа имеет ту же цель, и поэтому воспользуемся этим приемом для получения первого представления и навыков работы в IDE STUDIO на примере создания для МК Atmega128A программы вывода информации на светодиодную шкалу через параллельный порт.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 2.1.



Рис. 2.1. Состояние тумблеров конфигурации стенда

2.1. Описание лабораторной установки и примеры программ

Простейшим индикатором состояния различных объектов может служить светодиодная шкала, и поэтому первичное изучение работы параллельного порта в режиме вывода информации производится на примере программирования процедуры вывода сигналов на светодиодную шкалу для формирования различных световых эффектов. Фрагмент схемы лабораторного стенда, показывающий подключение светодиодной шкалы к параллельному порту PORTC, показан на рис. 2.2.

Светодиодная шкала представляет собой набор из восьми светодиодов, объединенных в виде линейной шкалы. Резисторы, включенные последовательно со светодиодами шкалы, предназначены для ограничения тока до номинального значения. Их представленной схемы нетрудно видеть, что для зажигания любого светодиода шкалы достаточно в соответствующий разряд порта PORTC записать значение логической единицы. При этом на выводе разряда порта появляется напряжение, зажигающее выбранный светодиод.

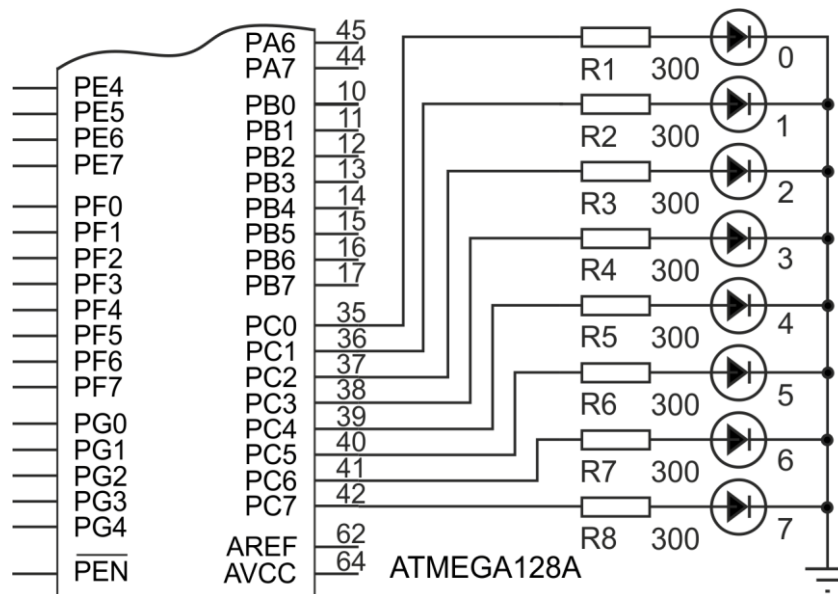


Рис. 2.2. Схема подключения светодиодной шкалы

Приведем пример программы, выполняющей подобное действие.

```
##include <avr/io.h>    // включить заголовочный файл
#define F_CPU 11059200 // задать частоту тактирования
#include <util/delay.h>

int main(){              // основная программа
    DDRC = 0xFF;          // все выходы порта PORTC - выходы
    PORTC = 0x00;         // погасить все светодиоды
    PORTC = PORTC | 1 << 0; // зажечь светодиод 1
    while(1);             // бесконечный цикл
}
```

Назначение операторов программы описано в комментариях, однако некоторые из них требуют уточнений.

В программе используются символьные имена регистра данных порта PORTC и регистра направления DDRC и они должны быть сопоставлены соответствующим этим регистрам физическим адресам с помощью описаний

```
sfrb DDRC=0x14;
sfrb PORTC=0x15;
```

Хотя в нашей программе этих описаний нет, но, тем не менее, при компиляции никаких ошибок выявлено не будет. Связано это с тем, что все символические имена регистров ввода-вывода определены в специальных заголовочных файлах, поставляемых с STUDIO для всех поддерживаемых им МК. Вся информация из этих файлов включается в программу с помощью директивы **#include**, что и сделано в первой строке нашей программы. Файл

avr/io.h и содержит сопоставление символических имен с адресами и другую информацию, относящуюся к МК семейства AVR.

Организация бесконечного цикла (в данном случае с помощью оператора **while**) характерна для большинства программ встроенных систем. Дело в том, что при отсутствии подобного цикла МК, выбрав из памяти последнюю команду программы и закончив ее выполнение, автоматически занесет в счетчик команд адрес следующей ячейки памяти программ, выберет хранящийся в ней код и начнет его выполнение. Естественно, что подобная попытка выполнения кода, не относящегося к рабочей программе, чревата непредсказуемыми последствиями.

В общем случае при написании программ следует использовать структурный подход, при котором часто повторяющиеся или самостоятельные фрагменты программы оформляются в виде функций. При таком подходе структура нашей программы может быть представлена в следующем виде:

```
#include <avr/io.h>    // включить заголовочный файл
#define F_CPU 11059200 // задать частоту тактирования
#include <util/delay.h>

#define D_LED DDRC      // рабочий порт направления
#define P_LED PORTC     // рабочий порт данных

void led (uint8_t n) {   // определение функции
    //[]-----[]
    //| Назначение: зажигание светодиода с номером n |
    //[]-----[]
    D_LED = 0xFF;        // все выходы P_LED - выходы
    P_LED = P_LED | (1 << n); // зажечь светодиод n
}

int main(){              // основная программа
    P_LED = 0x00;        // погасить все светодиоды
    led (0);             // зажечь светодиод 0
    while(1);            // бесконечный цикл
}
```

Естественно, что данная программа настолько проста, что использование в ней функции никаких преимуществ не дает, но в более сложных программах такой подход упрощает реализацию алгоритма функционирования и значительно облегчает процедуру отладки.

При выполнении заданий необходимо учитывать, что скорость выполнения команд МК слишком высока, и для визуального восприятия включенного и выключенного состояний светодиода следует вводить паузу между моментами переключения. В наиболее простом случае функция

временной задержки может представлять собой пустой цикл, выполняющийся заданное число раз. Однако задать точное время задержки, реализуемой таким способом, довольно трудно. Более рациональным является использование таймера, имеющегося в практически любом МК, с помощью которого легко задать точное время задержки в широком временном диапазоне.

В IDE STUDIO в стандартной библиотеке имеются две функции задания временных задержек (заметим только, что в первой из них задержка может быть задана только числовым аргументом)

```
void _delay_us(uint16_t n); // задержка в микросекундах
void delay_ms(uint16_t n); // задержка в миллисекундах
```

Прототипы данных функций заданы в заголовочном файле `delay.h`, и при их использовании в программе необходимо подключить этот файл с помощью директивы

```
#include <util/delay.h>
```

Составим теперь нашу первую тестовую программу, которая будет выполнять в бесконечном цикле следующие действия:

1. погасить все светодиоды;
2. сделать паузу 1 с;
3. зажечь нечетные светодиоды 1-3-5-7;
4. сделать паузу 1 с;
5. перейти к п. 1.

При составлении программы полагаем, что для зажигания комбинации светодиодов 1-3-5-7 в порт PORTC должна быть занесено двоичное число 0b10101010 (0xAA в шестнадцатеричном представлении).

Листинг программы, написанной с учетом заданного алгоритма, может быть представлен в следующем виде:

```
#include <avr/io.h> // включить заголовочный файл
#define F_CPU 11059200 // задать частоту тактирования
#include <util/delay.h>

// определение функций
void Bar(uint8_t led) {
    //[]-----[
    //| Назначение: зажигание светодиодов шкалы |
    //| led - комбинация зажигаемых светодиодов |
    //[]-----[
    DDRC = 0xFF; // все выводы PORTC - выходы
    PORTC = led // зажечь светодиоды
}
```

```

int main() {                                // основная программа
while(1) {                                  // бесконечный цикл
    Bar(0x00);                              // погасить все светодиоды
    _delay_ms(1000);                        // пауза 1 секунда
    bar(0xAA);                              // зажечь комбинацию
    _delay_ms(1000);                        // пауза 1 секунда
}
}

```

2.2. Варианты заданий

После успешного запуска первой программы, модифицируйте ее в соответствии с заданным преподавателем вариантом задания из перечисленного списка.

1. Реализуйте на светодиодной шкале эффект «бегущего огня». Для этого в бесконечном цикле последовательно зажигайте и гасите каждый светодиод шкалы с задержкой 1 с.

2. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с пары светодиодов 0-7, 1-6, 2-5, 3-4. После коммутации последней пары введите трехсекундную задержку.

3. В бесконечном цикле поочередно зажигайте и гасите с задержкой 1 с четверки светодиодов 0-3, 4-7.

4. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с светодиоды 0-2-4-6, 1-3-5-7.

5. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с тройки светодиодов 0-1-2, 1-2-3, 2-3-4, 3-4-5, 4-5-6, 5-6-7. После коммутации последней тройки введите трехсекундную задержку.

6. В бесконечном цикле зажигайте и гасите с задержкой 1 с пары светодиодов 3-4, 2-5, 1-6 и 0-7. После коммутации последней пары введите трехсекундную задержку.

7. В бесконечном цикле последовательно зажигайте все светодиоды шкалы с задержкой 1 с. После зажигания последнего светодиода погасите все светодиоды и повторите процедуру в обратном порядке.

8. В бесконечном цикле поочередно зажигайте и гасите с задержкой 1 с четверки светодиодов 0-2-4-6 и 1-3-5-7.

9. В бесконечном цикле последовательно зажигайте и гасите с задержкой 1 с пары светодиодов 0-1, 1-2, 2-3, 3-4, 4-5, 5-6 и 6-7. После коммутации последней пары введите трехсекундную задержку.

10. В бесконечном цикле последовательно зажигайте каждый светодиод шкалы с задержкой 1 с. После зажигания последнего светодиода погасите все светодиоды и повторите процедуру.

11. В бесконечном цикле зажгите все светодиоды и затем последовательно гасите каждый светодиод шкалы с задержкой 1 с. После гашения последнего светодиода повторите процедуру.

12. В бесконечном цикле зажгите все светодиоды и затем последовательно гасите пары светодиодов 0-1, 2-3, 4-5, 6-7 шкалы с задержкой 1 с. После гашения последней пары повторите процедуру.

3. Лабораторные работы

3.1. Содержание отчета

Отчет по лабораторной работе должен содержать следующие разделы:

- задание на лабораторную работу;
- принципиальную схему части макета, относящейся к заданию;
- блок-схему алгоритма программы;
- текст программы на языке C с обязательными комментариями.

3.2. Программирование процедур ввода-вывода информации с использованием параллельных портов

Данные, представленные в виде двоичных логических сигналов, являются одним из типов данных, наиболее часто используемых МК для обмена информацией с внешними объектами. Поэтому практически каждый МК имеет линии ввода-вывода таких данных, подключенные к внешним выводам. С помощью этих линий МК взаимодействует с внешними датчиками и управляет исполнительными устройствами.

Помимо обмена логическими данными через одиночные линии в МК используется и широко распространенный параллельный обмен информацией. Для этого одиночные линии ввода-вывода объединяются в параллельные восьмиразрядные порты ввода-вывода, подключенные к системной магистрали МК.

МК серии AVR в зависимости от модели имеют в своем составе от одного до семи параллельных портов ввода-вывода. Каждый разряд порта подсоединен к одному из выводов (ножек) микросхемы.

Порты являются двунаправленными, т.е. любую линию каждого порта можно независимо настроить как на ввод, так и на вывод. Каждый порт имеет в своем составе три расположенных в пространстве ввода-вывода регистра: регистр хранения выводимых данных PORTx, регистр направления данных DDRx и регистр входных данных PINx. Регистр входных данных PINx обеспечивает только возможность чтения, фактически он считывает информацию напрямую с вывода МК. Регистры данных PORTx и направления данных DDRx обеспечивают возможность и чтения и записи.

В том случае, если линия настроена на ввод, то для повышения помехоустойчивости имеется возможность подключения к линии ввода внутренних резисторов, подтягивающих ее к напряжению питания.

Содержимое регистров направления $DDRx$ (Data Direct Register) определяет направление передачи данных по каждой линии порта. Если бит регистра $DDRx$ установлен в 0, то соответствующая ему линия порта настраивается на ввод, в противном случае – линия используется для вывода цифровой информации.

Если линия порта настроена на вывод данных, то ее состояние будет определяться содержимым соответствующего разряда регистра $PORTx$.

Если же линия порта настроена на ввод данных, то установка в 1 соответствующего бита в регистре $PORTx$ приведет к подключению к этой линии подтягивающего резистора.

Состояние линий порта, в зависимости от комбинации содержимого разрядов регистров $DDRx$ и $PORTx$, приведено в табл. 3.1.

Таблица 3.1

$DDRx$	$PORTx$	Вх./Вых.	Состояние линии
0	0	Вход	Третье состояние (Hi-Z)
0	1	Вход	Режим входа с подтягивающим резистором
1	0	Выход	Вывод логического нуля
1	1	Выход	Вывод логической единицы

Упрощенная схема одного разряда любого порта $PORTxn$ представлена на рис. 3.1.

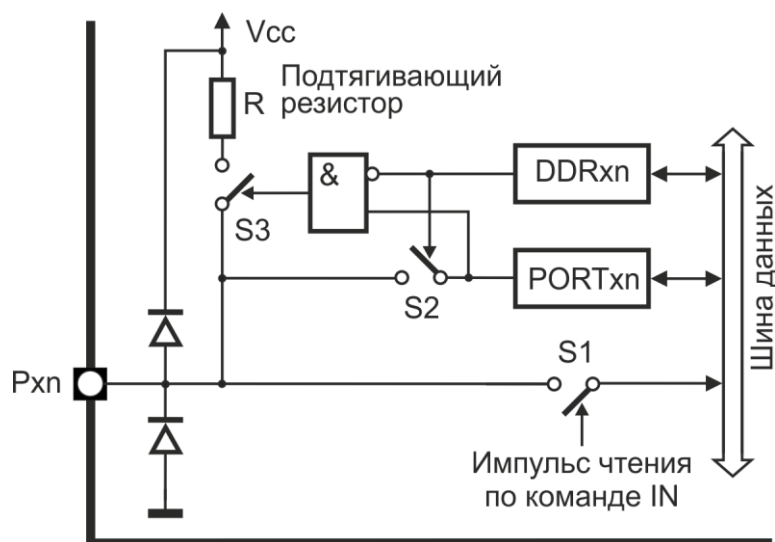


Рис. 3.1. Упрощенная схема одного разряда порта

Если разряд регистра направления $DDRxn$ установлен в 1, то ключ $S2$ замкнут, разряд $PORTxn$ порта настроен на вывод данных и состояние порта передается на ножку МК Pxn .

Для настройки заданного разряда порта PORT_{xn} на ввод данных соответствующий разряд регистра направления DDR_{xn} должен быть сброшен в 0. Это приводит к размыканию ключа S2 и разряд порта PORT_{xn} отключается от вывода P_{xn} и его состояние не искажает считываемых данных.

Для считывания входных данных используется команда IN, при выполнении которой формируется импульс чтения, замыкающий ключ S1. При этом вводимые данные не сохраняются в разрядах порта, а считываются непосредственно с контактов P_{xn} в шину данных. Если разряд DDR_{xn} = 0 и разряд PORT_{xn} = 1, то на выходе элемента «И-НЕ» устанавливается уровень логической 1, замыкающий ключ S3, с помощью которого подтягивающий резистор R подключается к внешнему выводу P_{xn}. Для всех остальных комбинаций на выходе элемента «И-НЕ» устанавливается уровень логического 0 и резистор будет отключен.

Если разряд DDR_{xn} = 0 и разряд PORT_{xn} = 0, то вывод порта P_{xn} находится в отключенном состоянии (все три ключа разомкнуты).

После сброса МК или подачи напряжения питания, начальное значение всех регистров направления DDR_x равно 0x00 и по умолчанию все порты настроены на ввод, а начальное значение любого регистра PORT_x также равно 0x00 и по умолчанию все разряды портов будут находиться в состоянии высокого импеданса.

В заключение раздела рассмотрим несколько примеров.

Пример 1.

```
DDRA = 0xFF; //настройка всех выводов порта A как выходов
PORTA = 0x0F; //вывод в разряды порта A значений 0b00001111
```

Пример 2.

```
uint8_t a;
DDRD = 0; //настройка всех выводов порта D как входов
i = PIND; //прочитать 8 выводов порта D в переменную i
```

Пример 3.

```
// настроить разряды 1, 3, 5 порта C на выход
DDRC = DDRC | ((1 << 1) | (1 << 3) | (1 << 5));
// вывести в разряды 1, 3, 5 порта C логические 1
PORTC = PORTC | ((1 << 1) | (1 << 3) | (1 << 5));
```

Лабораторная работа № 1

Цель работы. Программирование процедур вывода информации на семисегментный индикатор с использованием параллельного порта.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.4.



Рис. 3.2. Состояние тумблеров конфигурации стенда для работы 1

Методические указания по выполнению работы

Для отображения цифровой и простейшей символьной информации в микропроцессорных системах очень часто используются светодиодные сборки, называемые семисегментными индикаторами. Как следует из его названия, индикатор состоит из семи сегментов, ориентированных таким образом, что подсветка различных комбинаций сегментов позволяет формировать алфавитно-цифровые символы. Каждый сегмент подсвечивается своей цепочкой светодиодов с возможностью индивидуального включения сегмента. Поскольку в первую очередь индикатор предназначен для отображения цифр, то в его состав включена также десятичная точка. Внешний вид и схема включения светодиодов всех сегментов индикатора показаны на рис. 3.3.

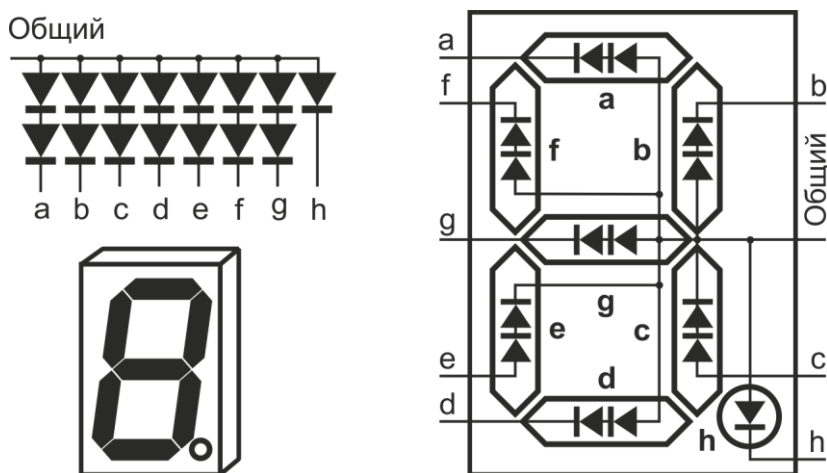


Рис. 3.3. Семисегментный индикатор

В лабораторном стенде используется набор из пяти семисегментных индикаторов, позволяющий выводить пятиразрядные числа. Каждый разряд такого числа последовательно, один за другим, выводится с помощью одного порта МК на соответствующий индикатор.

Однако для вывода числа на индикатор необходимо подавать на него не код числа, а код его отображения на индикаторе. Например, для вывода числа 3 необходимо подать высокий уровень сигнала на входы a, b, c, d и g.

В случае, когда сегменты индикатора a ÷ g подключены к разрядам используемого порта МК 0 ÷ 7 соответственно, то все коды отображения чисел на индикаторе можно представить в виде, сведенном в табл. 3.1. Последние строки в таблице необходимо заполнить самостоятельно.

Таблица 3.1.

№ ножки	7	6	5	4	3	2	1	0	Код
Сегмент	h	g	f	e	d	c	b	a	отображения
Цифра «0»	0	0	1	1	1	1	1	1	0x3F
Цифра «1»	0	0	0	0	0	1	1	0	0x06
Цифра «2»	0	1	0	1	1	0	1	1	0x5B
Цифра «3»	0	1	0	0	1	1	1	1	0x4F
Цифра «4»	0	1	1	0	0	1	1	0	0x66
Цифра «5»	0	1	1	0	1	1	0	1	0x6D
Цифра «6»	0	1	1	1	1	1	0	1	0x7D
Цифра «7»	?	?	?	?	?	?	?	?	?
Цифра «8»	?	?	?	?	?	?	?	?	?
Цифра «9»	?	?	?	?	?	?	?	?	?

Для организации процесса последовательного вывода разрядов отображаемого числа и управления им используются специальные микросхемы – восьмиразрядные регистры-защелки 74HC573. Функция такой микросхемы состоит в том, что при подаче на вход LE (Latch Enable) управляющего сигнала высокого уровня в ее внутренний регистр записывается комбинация восьми входных сигналов D0...D7, затем, после подачи на LE сигнала низкого уровня, входные линии отключаются и введенная комбинация сохраняется (защелкивается) во внутреннем регистре. При подаче на второй управляющий вход OE (Output Enable) сигнала низкого уровня, эта предварительно запомненная комбинация входных сигналов выводится на выходные линии микросхемы-защелки Q0...Q7. В схеме лабораторного стенда ножка OE

заземлена, то есть всегда имеет низкий уровень, и, следовательно, управление производится только уровнем сигнала на выводе LE.

Схема подключения индикаторов к МК с использованием регистров-защелок показана на рис. 3.4.

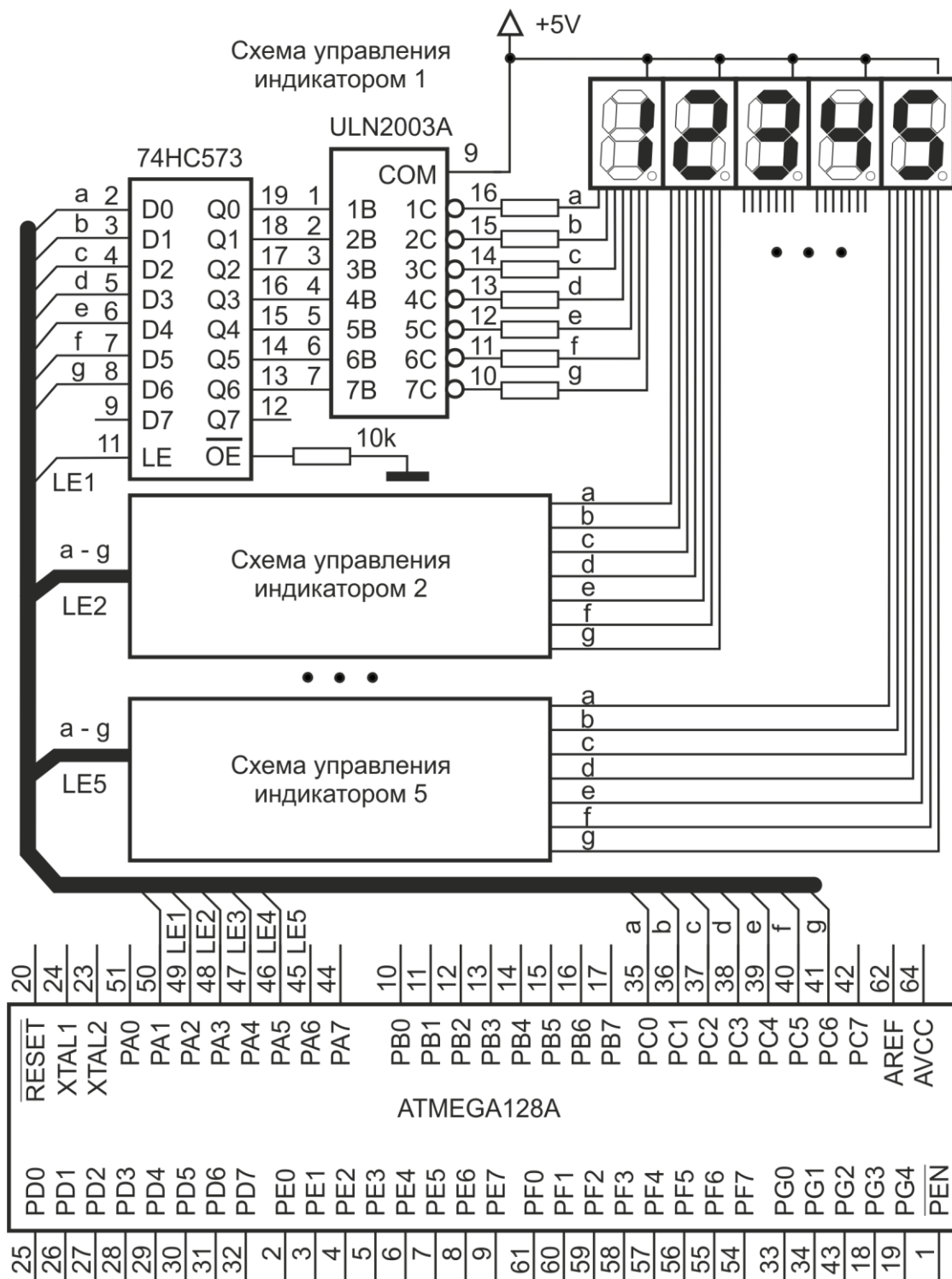


Рис. 3.4. Схема включения семисегментных индикаторов

В стенде используются семисегментные индикаторы, в каждом сегменте которых расположена цепочка из нескольких последовательно соединенных светодиодов, вследствие чего для зажигания сегмента требуется ток в несколько десятков миллиампер, недопустимый для выходных линий регистра-защелки. Поэтому в схеме предусмотрено включение между регистром-защелкой и индикатором силового драйвера – микросхемы ULN2003, предназначенной для подключения устройств с сильноточной нагрузкой. В данном случае драйвер является инвертирующим – при подаче на его вход логической единицы на выходе формируется логический ноль. Графически инверсия иллюстрируется наличием небольшого кружка на выходной линии драйвера.

Как видно из схемы, входные линии всех защелок подключены к линиям параллельного порта PORTC, через который выводится код отображаемого символа. Входы LE регистров-защелок 1...5 подсоединены к линиям PA1...PA5 порта PORTA соответственно. Такое включение индикаторов позволяет последовательно выводить через порт PORTC код символа отображаемого разряда десятичного числа, затем стробом 0-1-0, подаваемым в одну из линий порта PA1...PA5 запоминать этот код в соответствующем отображаемому разряду регистре-защелке. После окончания процедуры заполнения регистров-защелок для всех пяти индикаторов на каждом из них будет отображен символ, код которого был ранее сохранен в его защелке.

Из логики работы используемой схемы включения следует, что вывод на индикатор с пятью десятичными разрядами любого целого числа производится поразрядно и для выполнения этой операции необходимо предварительно выделить значение цифры в каждом разряде выводимого числа.

Вычислить значения цифр в разрядах пятиразрядного десятичного числа поможет следующая функция, которую можно без изменений использовать в разрабатываемых программах.

```
uint8_t Digit (uint16_t d, uint8_t m){
    //[ ]-----[ ]
    //| Назначение: выделение цифр из разрядов пятиразрядного |
    //| десятичного положительного числа |
    //| Входные параметры: |
    //| d - целое десятичное положительное число |
    //| m - номер разряда (от 1 до 5, слева направо) |
    //| Функция возвращает значение цифры в разряде m числа d |
    //[ ]-----[ ]
```

```

uint8_t i = 5, a;
while(i){                                // цикл по разрядам числа
    a = d%10;                             // выделяем очередной разряд
    if(i-- == m) break;                   // выделен заданный разряд - уходим
    d /= 10;                              // уменьшаем число в 10 раз
}
return(a);
}

```

Примеры использования функции:

```

uint8_t k;
uint16_t f = 7236;
k = Digit(f,5);    // функция возвращает k = 6
k = Digit(f,2);    // функция возвращает k = 7
k = Digit(f,1);    // функция возвращает k = 0

```

Поскольку двоичный код отображаемой цифры не совпадает с ее байтом индикации, то в программе должна осуществляться перекодировка в соответствии с таб. 3.2, ставящей в соответствие двоичный код индицируемой цифры своему байту индикации, который и выдается в выходной порт для отображения цифры на индикаторе.

С учетом сказанного, для формирования цифр, отображаемых на пятиразрядном семисегментном индикаторе стенда, нужно выполнить для $k = 1 \div 5$ следующую последовательность действий:

1. с помощью функции Digit получить цифру k-го десятичного разряда выводимого числа;
2. по информации, представленной в таб. 2.2, сформировать код отображения полученной цифры;
3. вывести код отображения в порт PORTC;
4. защелкнуть код отображения в регистре-защелке k, для чего подать сигнал $LE = 1$, записав 1 в разряд k порта PORTA, сделать паузу 1 мкс и подать 0 для выдачи сигнала $LE = 0$;
5. если $k = 5$, то закончить, иначе – перейти к п. 1.

Задание на работу №1: рассчитать и с паузой в 1 сек вывести последовательно на индикатор 20 членов числовой последовательности, сформированной по формуле арифметической прогрессии

$$x(i+1) = x(i) + d$$

для вариантов заданий, представленных в таблице.

№ варианта	$x(0)$	d	№ варианта	$x(0)$	d
1	1	4	7	3	3
2	2	3	8	4	4
3	3	2	9	4	2
4	4	1	10	3	1
5	1	1	11	2	3
6	2	2	12	1	4

Лабораторная работа № 2

Цель работы. Работа с системой внешних прерываний на микроконтроллере на примере инкрементального энкодера.

В общем случае запрос на прерывание во время работы МК приводит к временной остановке выполнения основной программы и передаче управления специальной подпрограмме обслуживания прерывания (ПОП), в которой реализуется реакция на событие, вызвавшее запрос прерывания. По окончании работы ПОП выполнение основной программы должно возобновляться с того момента, в который она была прервана по запросу прерывания. Любой запрос на прерывание вызывает установку флага прерывания, соответствующего источнику прерывания, вызвавшего запрос.

В разных средах программирования объявление функций обработчиков прерываний сделано по-разному. В Studio объявление функции обработчика прерывания осуществляется следующим образом:

```
ISR (INT7_vect)
{
    /* Это функция-обработчик прерывания. Здесь пишутся команды, которые выполняются, когда произойдет событие на ножке INT7 */
}
```

Для того чтобы функция обработчик прерывания была вызвана необходимо выполнение трёх условий:

- 1) Прерывания должны быть разрешены глобально, делается это установкой в «1» бита I регистра SREG. Для этого выделена специальная машинная команда контроллера – **sei**. Для сброса этого бита также выделена отдельная команда – **cli**. Для использования ассемблерных команд в Studio используется функция **sei()**. Таким образом, для глобального разрешения прерываний необходимо добавить строчку **sei()** в одну из функций программы, например в функцию **main**.
- 2) Должно быть разрешено соответствующее прерывание, делается это с помощью конфигурации соответствующих регистров.
- 3) Должно произойти событие вызывающее соответствующее прерывание, например переполнение таймера или изменение сигнала на входе внешнего прерывания.

По умолчанию при входе в обработчик прерывания контроллер глобально запрещает прерывания и разрешает их при выходе из функции обработчика. Делается это с целью недопущения «зависания» контроллера в процессе обработки прерывания.

При работе с прерываниями необходимо помнить некоторые важные моменты:

- 1) Во время выполнения функции обработчика прерывания основная программа не выполняется, поэтому при большой частоте прерываний основная программа может вообще остановить своё выполнение.
- 2) При обращениях в основной программе к переменным, изменяемым в обработчике прерываний и имеющим размер больше одного байта, должны быть запрещены прерывания изменяющие её. При работе с такими переменными прерывание может случиться между командами, считывающими разные байты переменной, что приведёт к неправильному чтению или записи.
- 3) Каждый источник прерываний имеет только один флаг, указывающий на то, что произошло событие вызывающее это прерывание. Поэтому если до вызова обработчика прерывания случиться два и более таких событий, обработчик всё равно будет вызван только один раз.

Для работы с внешними прерываниями в МК AVR есть несколько регистров. Каждое внешнее устройство прерывания подключается к контактам внешних прерываний МК (контакты внешних прерываний Atmega128 $PD0...PD7$, см. $INT0...INT7$ на рис. 1.3). Если пользователю требуется работать с внешним источником прерывания (например, кнопкой), необходимо установить вывод МК, к которому подключена кнопка, как вывод внешнего прерывания (а не как контакт ввода/вывода). Для этого используется регистр EIMSK.

Бит	7	6	5	4	3	2	1	0	
	INT7	INT6	INT5	INT4	INT3	INT2	INT1	INT0	EIMSK
Н.з.	0	0	0	0	0	0	0	0	

Чтобы разрешить прерывания на i -ом выводе INT_i МК, необходимо в соответствующий i -ый бит регистра EIMSK поместить логическую единицу. Для работы с ножками INT_i как с ножками ввода/вывода, необходимо в соответствующий i -ый бит регистра EIMSK поместить логический ноль.

Событие для запуска процедуры обработчика прерывания может быть установлено с помощью регистров EICRA и EICRB. Рассмотрим кнопку, кото-

Бит	7	6	5	4	3	2	1	0	
	ISC31	ISC30	ISC21	ISC20	ISC11	ISC10	ISC01	ISC00	EICRA
Н.з.	0	0	0	0	0	0	0	0	

рая коммутирует сигнал GND на некоторую ножку прерывания МК, которая по умолчанию подтянута к питанию. Очевидно, что при нажатии кнопки напряже-

ние на выводе $INTi$ МК переключается с 1 на 0 (с VCC на GND), а при отпуске кнопки напряжение на выводе $INT0$ МК переключается с 0 на 1 (с GND на VCC). Итак, для выбора события (с 1 на 0, с 0 на 1 и т.д.) запуска процедуры обработчика прерывания используются EICRA и EICRB.

Бит	7	6	5	4	3	2	1	0	
	ISC71	ISC70	ISC61	ISC60	ISC51	ISC50	ISC41	ISC40	EICRB
Н.з.	0	0	0	0	0	0	0	0	

EICRA работает с выводами внешних прерываний $INT0... INT3$, EICRB работает с выводами внешних прерываний $INT4... INT7$.

Комбинации EICRA

ISCn1	ISCn0	Описание
0	0	Низкий уровень на $INTn$ генерирует запрос прерывания
0	1	Зарезервировано
1	0	Падающий срез на $INTn$ генерирует запрос прерывания
1	1	Нарастающий фронт на $INTn$ генерирует запрос прерывания

Комбинации EICRB

ISCn1	ISCn0	Описание
0	0	Низкий уровень на $INTn$ генерирует запрос прерывания
0	1	Любое изменение (1-в-0 или 0-в-1) на $INTn$ генерирует запрос
1	0	Падающий срез на $INTn$ генерирует запрос прерывания
1	1	Нарастающий фронт на $INTn$ генерирует запрос прерывания

Запрос прерывания – это событие, которое происходит, если внешний или внутренний источник прерывания разрешен локально (в случае внешнего прерывания – редактируется EIMSK) и правильно настроен (в случае внешнего прерывания – редактируются EICRA/EICRB). Кроме того, прерывания должны быть разрешены глобально инструкцией **sei()**.

Механический энкодер предназначен для ввода информации (например, для плавного изменения параметров), как более надежная и удобная замена потенциометру. В отличие от потенциометра энкодер не имеет ограничения угла поворота.

Принцип работы энкодера представлен на рис. 3.5. При вращении энкодера на

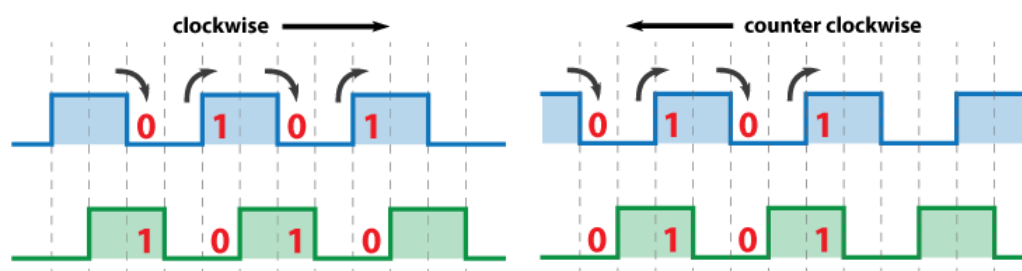


Рис. 3.5. Принцип работы механического энкодера

его выходах А и В (PD0 (INT0) и PD1 (INT1) контроллера, соответственно) формируются импульсы. Если вращать энкодер вправо, то импульс А будет незначительно опережать импульс В, если вращать влево, то - наоборот.

Таким образом, считая импульсы на любом из выходов можно определить на какой угол повернули энкодер, а по последовательности выходов можно определить направление вращения.

Подключение энкодера приведено на рис. 3.6.

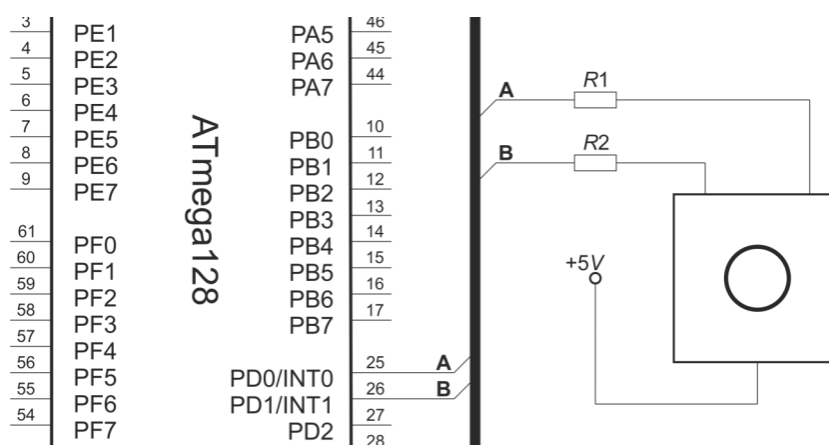


Рис. 3.6. Схема для 1-ой работы

Для определения угла поворота энкодера удобно использовать одно внешнее прерывание. В этом случае необходимо в обработчике прерывания по изменению уровня на одном из выходов энкодера проверять второй выход энкодера и по его значению увеличивать или уменьшать переменную, хранящую текущий угол поворота. В нашем случае задача усложняется ввиду того что внешние прерывания 0 и 1 невозможно настроить на изменение по любому изменению уровня, поэтому в самом обработчике прерывания необходимо определять текущее состояние ножки и настраивать прерывание на срабатывание по изменению состояния на противоположное, например, если на ножке прерывания высокий сигнал, то в обработчике его необходимо перенастроить на срабатывание по срезу. Пример процедуры обработчика прерывания представлен ниже.

```
ISR (INT0_vect)
{
    if((PIND & (1 << 0)) != 0)
    {
        EICRA = (1<<ISC01);
        if((PIND & (1 << 1)) != 0) angle++;
        else angle--;
    }
    else
    {
```

```

        EICRA = (1 << ISC01) | (1<<ISC00);
        if((PIND & (1 << 1)) != 0) angle--;
        else angle++;
    }
}

```

где angle - это глобальная переменная типа `uint16_t`, хранящая текущее значение поворота энкодера.

В работе, помимо механического энкодера, также принимает участие модуль семисегментных индикаторов, схема подключения которого приведена на рис. 3.4 (см. лабораторную работу №1).

Для работы с семисегментными индикаторами можно использовать следующие функции:

```

// Массив кодов цифр
const uint8_t segments[] = {0x3F, 0x06, 0x5B, 0x4F,
0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

uint8_t Digit (uint16_t d, uint8_t m){
//[]-----[]
//| Назначение: выделение цифр из разрядов пятиразрядного|
//| десятичного положительного числа                      |
//| Входные параметры:                                     |
//|   d - целое десятичное положительное число             |
//|   m - номер разряда (от 1 до 5, слева направо)        |
//| Возвращается значение цифры в разряде m числа d       |
//[]-----[]
uint8_t i = 5, a;
while(i){           // цикл по разрядам числа
    a = d%10;        // выделяем очередной разряд
    if(i-- == m) break; // выделен заданный разряд - уходим
    d /= 10;         // уменьшаем число в 10 раз
}
return(a);
}

```

Задание на работу №2: написать программу вывода значения поворота энкодера на семисегментные индикаторы, используя внешнее прерывание №1 для определения направления вращения энкодера.

Лабораторная работа № 3

Цель работы. Программирование процедур вывода информации на светодиодную матрицу с использованием параллельного порта.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.7.



Рис. 3.7. Состояние тумблеров конфигурации стенда для работы 3

Методические указания по выполнению работы

Светодиодная матрица (Dot Matrix) представляет собой прямоугольный корпус с набором точек, каждая из которых состоит из одного светодиода. С помощью такой матрицы можно легко формировать алфавитно-цифровые символы, а также всевозможные пиктограммы. Внешний вид и схема используемой в работе матрицы приведены на рис. 3.8.

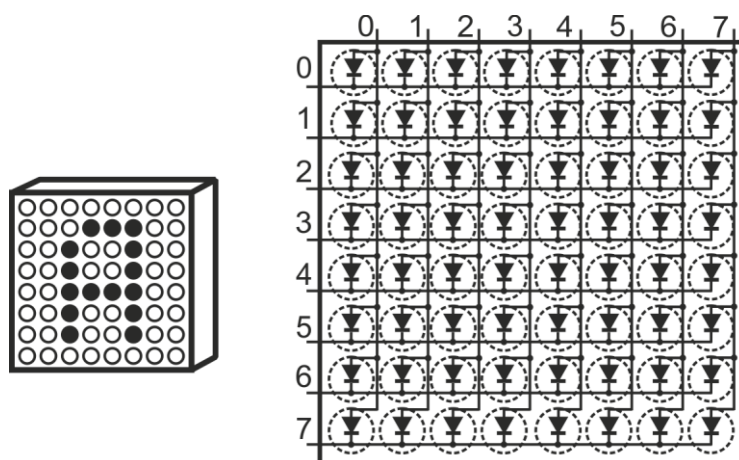


Рис. 3.8. Светодиодная матрица

В данной работе используется только одна светодиодная матрица из четырех, для которой номер активного столбца (8 линий) задается комбинацией четырех входных сигналов дешифратора, а выбор строки (8 линий) производится непосредственной подачей сигналов из порта. Такое включение и используется в стенде, фрагмент схемы которого показан на рис. 3.9.

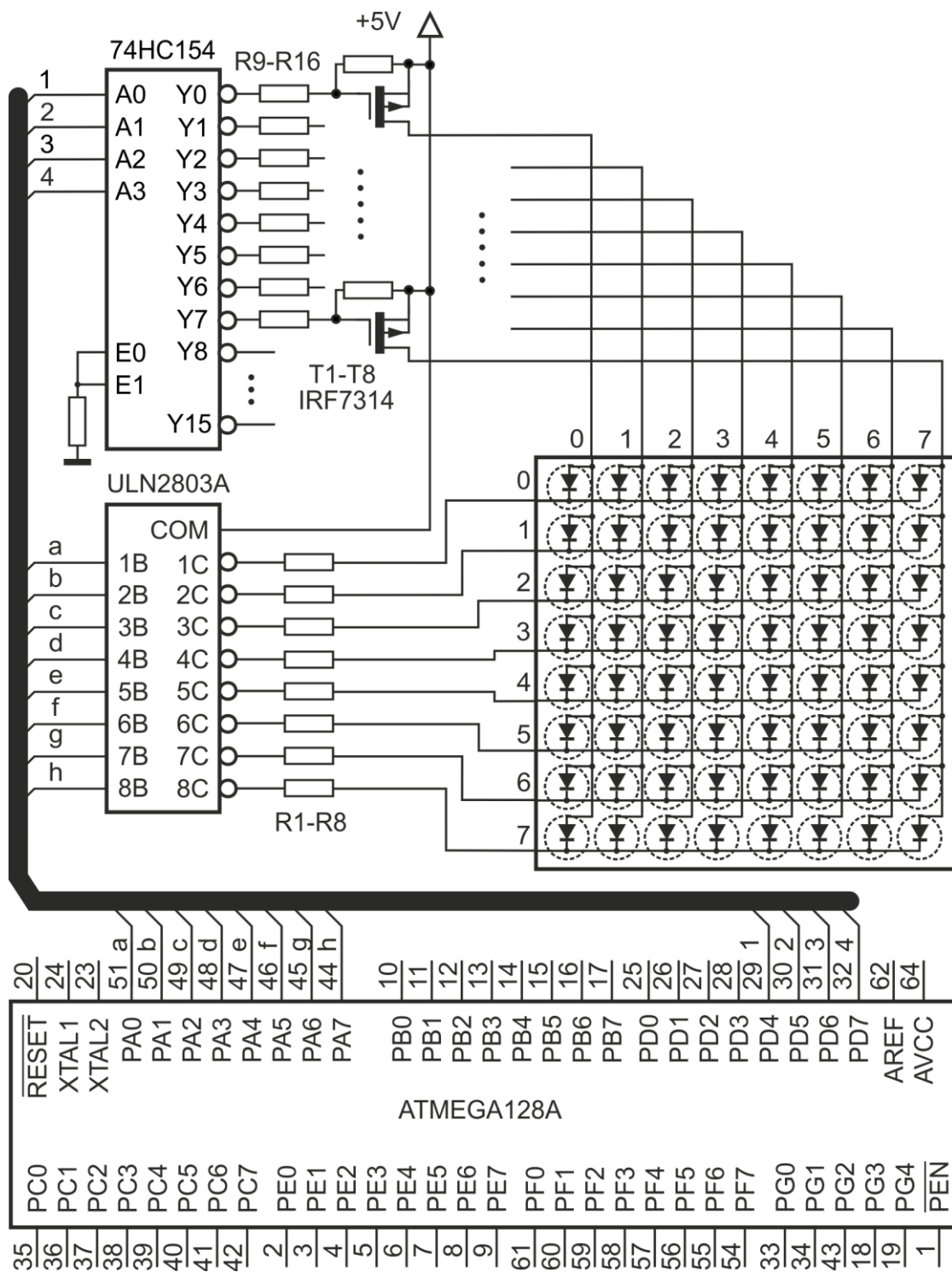


Рис. 3.9. Схема включения светодиодной матрицы

Поскольку нагрузочная способность разрядов порта для зажигания светодиодов недостаточна, то между ними и линиями столбцов матрицы включены силовые драйверы – микросхемы ULN2804A. При маломощных входных сигналах драйвер позволяет коммутировать на своем выходе

значительно более мощную нагрузку. Для линий строк в качестве усилителей использованы мощные полевые транзисторы.

В соответствии с принципом работы дешифратора задание на его входе номера выходной линии делает ее активной, т.е. равной 0 (поскольку выходы дешифратора инверсные). На всех остальных выходных линиях дешифратора устанавливается высокий уровень.

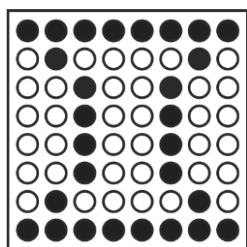
Вследствие этого в любой момент времени в матрице могут гореть светодиоды только одной строки, номер которой подан на вход дешифратора. Для того чтобы одновременно зажечь несколько светодиодов в строке, необходимо после активизации строки вывести в порт PORTA код, в котором зажигаемым точкам соответствует значение 1, а погашенным – 0.

Если смена строк с зажженными светодиодами будет меняться чаще, чем 25 раз в секунду, то в силу инерционности зрения будет наблюдаться одновременное свечение светодиодов всех строк, и, соответственно, всей матрицы в целом.

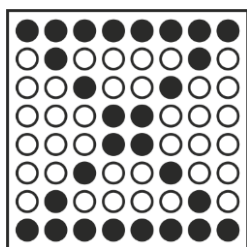
С учетом приведенного описания составим для примера фрагмент программы зажигания одной строки светодиодной матрицы.

```
DDRA = 0xFF;      // настроить порт PORTA на выход
DDRD = 0xF0;      // разряды PD4...PD7 порта PORTD - выходы
PORTD = 0x30;     // активизировать столбец 3
PORTA = 0xAA;     // зажечь в нем точки с нечетным номером
PORTD = 0x60;     // активизировать столбец 6
PORTA = 0x18;     // зажечь в нем две средние точки
```

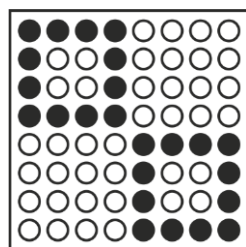
Задание на работу №3: составить программу вывода на светодиодную матрицу приведенных ниже пиктограмм в соответствии с заданным вариантом работы. После вывода заданной пиктограммы сделать паузу длительностью 3 с, произвести инверсию состояния всех точек и вновь сделать паузу. Все действия производить в бесконечном цикле.



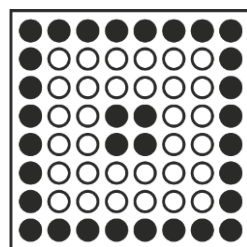
Вариант 1



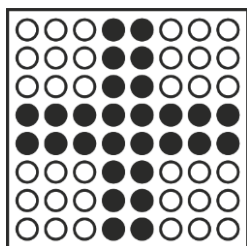
Вариант 2



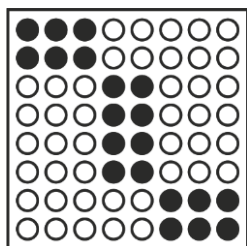
Вариант 3



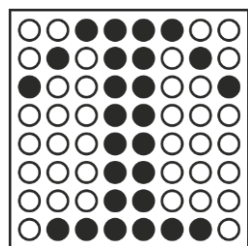
Вариант 4



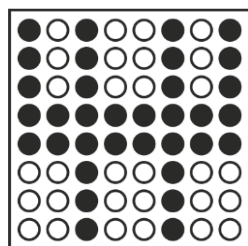
Вариант 5



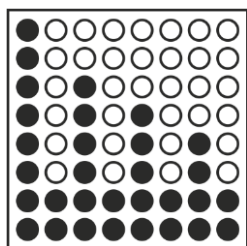
Вариант 6



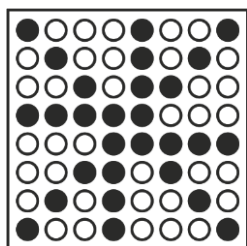
Вариант 7



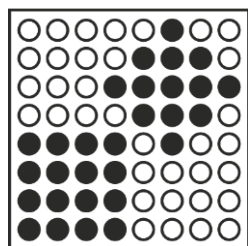
Вариант 8



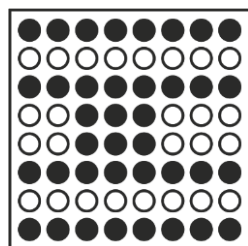
Вариант 9



Вариант 10



Вариант 11



Вариант 12

Лабораторная работа № 4

Цель работы. Изучение принципов работы внутренних таймеров/счётчиков МК.

Таймеры-счётчики предназначены для точного задания временных интервалов, генерации прямоугольных импульсов и измерения временных характеристик импульсных сигналов.

Микроконтроллер ATmega128A имеет четыре таймера-счётчика, два 8-ми битных (Таймеры №0 и №2) и два 16-ти битных (Таймеры №1 и №3). Каждый из таймеров имеет счётный регистр TCNTx (где x – номер таймера), соответствующего размера, значение в котором автоматически увеличивается с определённой, заранее заданной скоростью. Причём это увеличение происходит не зависимо от работы основной программы. Регистр 8-и битного таймера может хранить число из диапазона 0-255, а 16-и битного из диапазона 0-65535. При переполнении регистра его значение становится равным нулю, а также генерируется запрос на прерывание по переполнению, в виде установки флага. Таймер также может генерировать запрос на прерывание по совпадению значения его счётного регистра с заранее заданным числом, хранимым в регистре сравнения OCRx, 16-и битные таймеры имеют три независимых регистра сравнения (OCRxA, OCRxB, OCRxC, где x – номер таймера).

В данной лабораторной работе мы рассмотрим режим генерации ШИМ-сигнала с фиксированной частотой для 16-ти битного таймера.

Управление 16-битными таймерами/счётчиками. 16-битные таймеры Atmega имеют 3 независимых канала ШИМ – A , B и C . Для установки режима работы таймера используются регистры TCCRxA и TCCRxB ($x = 1$ или 3).

Bit	7	6	5	4	3	2	1	0	
	COMxA1	COMxA0	COMxB0	COMxB0	COMxC1	COMxC0	WGMx1	WGMx0	TCCRxA
Init.	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	ICNCx	ICESx	n/a	WGMx3	WGMx2	CSx2	CSx1	CSx0	TCCRxB
Init.	0	0	0	0	0	0	0	0	

Биты CSxi – это биты предделителя. Если необходимо замедлить таймер, пользователь может использовать преддлитель. В таблице ниже дано описание

битов CSx_i . Согласно ей, если все биты CSx_i равны нулю, таймер остановлен, это означает, что если пользователю нужно запустить таймер, биты CSx_i должны быть отредактированы.

$CSx2$	$CSx1$	$CSx0$	Описание
0	0	0	Таймер остановлен (!)
0	0	1	Предделитель на 1
0	1	0	Предделитель на 8
0	1	1	Предделитель на 64
1	0	0	Предделитель на 256
1	0	1	Предделитель на 1024
1	1	0	Внешний задатчик на ножке T_n . Счёт по срезу
1	1	1	Внешний задатчик на ножке T_n . Счёт по фронту

Биты $WGMx_i$ определяют режим генерации сигнала таймера/счетчика и задают верхний предел счёта (TOP). В таблице ниже показаны режимы работы 16-ти битных таймеров/счётчиков.

$WGMx3$	$WGMx2$	$WGMx1$	$WGMx0$	Режим таймера	Предел
0	0	0	0	Нормальный	0xFFFF
0	0	0	1	ШИМ с ФК, 8 бит	0x00FF
0	0	1	0	ШИМ с ФК, 9 бит	0x01FF
0	0	1	1	ШИМ с ФК, 10 бит	0x03FF
0	1	0	0	Сброс по совпадению (СТС)	OCRxA
0	1	0	1	Быстрая ШИМ, 8 бит	0x00FF
0	1	1	0	Быстрая ШИМ, 9 бит	0x01FF
0	1	1	1	Быстрая ШИМ, 10 бит	0x03FF
1	0	0	0	ШИМ ФЧК	ICRx
1	0	0	1	ШИМ ФЧК	OCRxA
1	0	1	0	ШИМ ФК	ICRx
1	0	1	1	ШИМ ФК	OCRxA
1	1	0	0	СТС	ICRx
1	1	0	1	Резерв	-
1	1	1	0	Быстрая ШИМ	ICRx
1	1	1	1	Быстрая ШИМ	OCRxA

Биты $COMxNi$ (где N – имя канала, $i = 0 \dots 1$) определяют вид выходного сигнала ($OCxN$). Если один или оба бита $COMxA1$: 0 выставлены в единицу,

выход ОСхА запрещает нормальное функционирование ножки порта вывода/ввода, к которому он подключен. То же самое для каналов *B* и *C*. Бит регистра направления данных (DDR), соответствующий контакту ОСхА, ОСхВ или ОСхС, должен быть выставлен в 1 для посылки генерируемого ШИМ сигнала во внешнее устройство. См. таблицы ниже.

Режим вывода при сравнении, не ШИМ

<i>COMxNI</i>	<i>COMxN0</i>	Описание
0	0	Нормальный режим, ОСхN отключен
0	1	Изменение состояния ОСхN при совпадении
1	0	Сброс ОСхN при совпадении
1	1	Установка ОСхN при совпадении

Режим вывода при сравнении, быстрая ШИМ

<i>COMxNI</i>	<i>COMxN0</i>	Описание
0	0	Нормальный режим, ОСхN отключен
0	1	Зарезервировано
1	0	Сброс ОСхN при совпадении
1	1	Установка ОСхN при совпадении

Режим вывода при сравнении, ШИМ с фазовой коррекцией

<i>COMxNI</i>	<i>COMxN0</i>	Описание
0	0	Нормальный режим, ОСхN отключен
0	1	Зарезервировано
1	0	Сброс ОСхN во время прямого счёта, установка во время обратного счёта
1	1	Установка ОСхN во время обратного счёта, сброс во время прямого счёта

В МК сигнал ШИМ формируется при сравнении значения регистра TCNT с заранее заданным значением из регистра OCR (см. рис. 3.10).

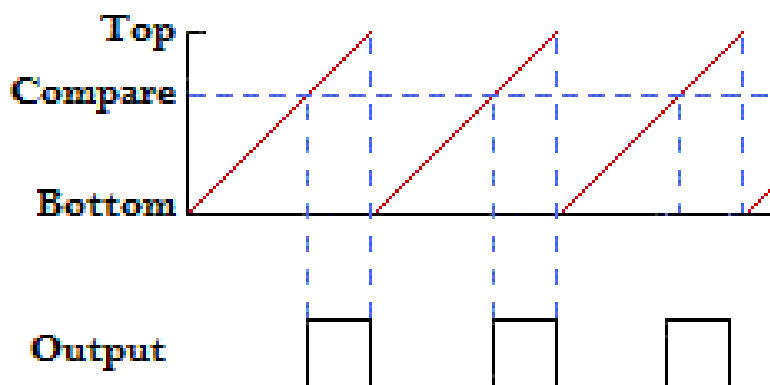


Рис. 3.10. ШИМ сигнал

Сравниваемое значение хранится в регистрах OCRxNH и OCRxNL (где N – имя канала).

Бит	15	14	13	12	11	10	9	8	
	OCRxN[15...8]								OCRxNH
Н.з.	0	0	0	0	0	0	0	0	

Бит	7	6	5	4	3	2	1	0	
	OCRxN[7...0]								OCRxNL
Н.з.	0	0	0	0	0	0	0	0	

Схема к лабораторной работе приведена на рис. 3.11.

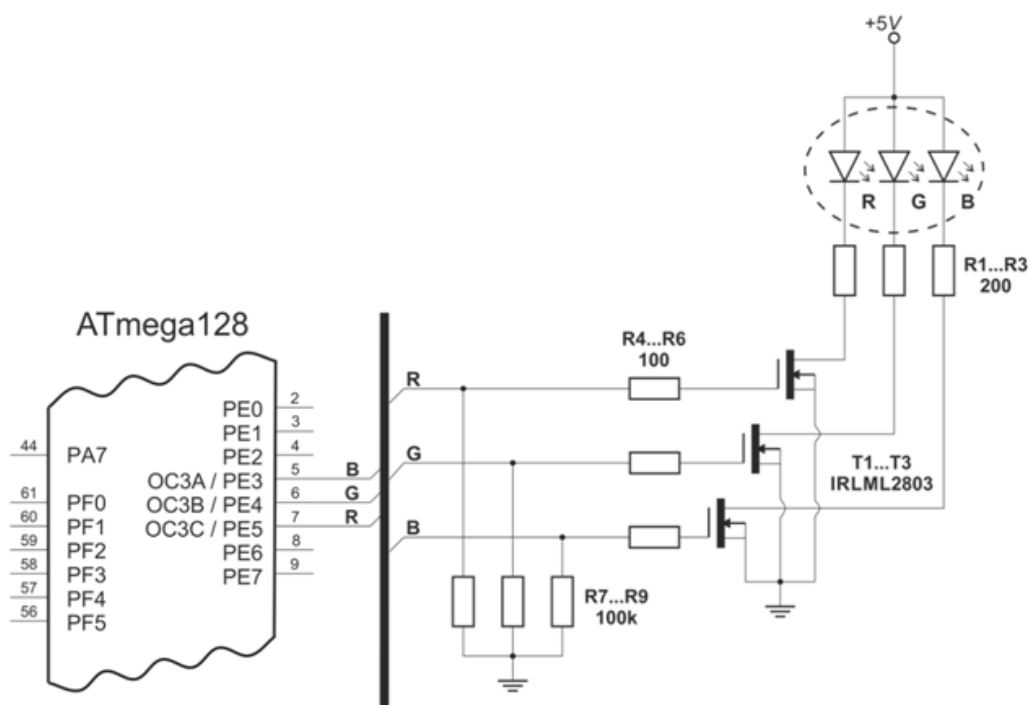


Рис. 3.11. Схема для 4-ой работы

Пример зажигания синего светодиода с постоянной яркостью представлен ниже:

```
int main(void)
{
    // Инициализация портов ввода/вывода
    DDRE = (1<<3) | (1<<4) | (1<<5);
    /* Инициализация таймера №3. 8-ми битная быстрая
    ШИМ, предделитель на 8 */
    TCCR3A = (1<<COM3A1) | (1<<COM3B1) | (1<<COM3C1) | (1<<WGM30);
    TCCR3B = (1<<WGM32) | (1<<CS31);
```

```

OCR3AH = 0;
OCR3AL = 128;
while(1) ;
}

```

Задание на работу №4: написать программу управления цветом и яркостью светодиода с помощью механического энкодера. В таблице (на следующей странице) заданий 0...255 означает, что при вращении энкодера почасовой стрелке яркость данного цвета увеличивается, а в левую – уменьшается.

Варианты заданий на работу

№№ Вар.	Описание исполняемого цикла: плавно изменять цвета		
	Красный цвет	Синий цвет	Зелёный цвет
1	0...255	0	0
2	0	0...255	0
3	0	0	255...0
4	0...255	0...255	0...255
5	0...255	0...255	0
6	0	255...0	0...255
7	255...0	0	255...0
8	255...0	0	0...255
9	0...255	0	0...255
10	0	255...0	255...0

Лабораторная работа № 5

Цель работы. Изучение принципов программного управления шаговым двигателем с использованием параллельного порта МК.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.12.



Рис. 3.12. Состояние тумблеров конфигурации стенда для работы 5

Методические указания по выполнению работы

Шаговый двигатель (ШД) является одним из широко используемых элементов современных систем автоматического управления. Он представляет собой бесколлекторный двигатель постоянного тока с фиксированными положениями вала. Просто подавая на двигатель питание, заставить его работать невозможно. Для его работы необходимо подавать на обмотки двигателя специальным образом сформированные последовательности импульсов, каждый из которых вызывает перемещение вала двигателя на заданный угол. Конструкция шагового двигателя и схема включения его обмоток показаны на рис. 3.13.

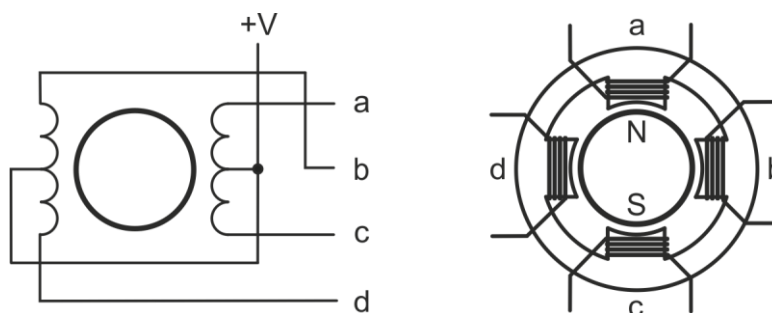


Рис. 3.13. Конструкция и схема включения обмоток ШД

В ШД управляющие импульсы подаются поочередно на обмотки статора, в результате чего ориентация магнитного потока в пространстве между полюсами статора дискретно меняется и вызывает поворот ротора на некоторый фиксированный угол, называемый шагом ротора. Для того чтобы заставить вал двигателя вращаться, необходимо поочередно подавать напряжение на обмотки статора (фазы) в нужной последовательности. Смена

направления вращения вала осуществляется простой заменой последовательности подключения обмоток на обратную, а изменение частоты вращения вала – изменением длительности управляющих импульсов.

В лабораторном стенде используется униполярный ШД с четырьмя обмотками, который имеет шаг 1.8° и, вследствие этого, для полного оборота вала необходимо подать на обмотки 200 импульсов.

Существует несколько способов управления ШД, каждый из которых может быть достаточно просто реализован программно с использованием управляющего МК. Рассмотрим эти способы управления более подробно.

Первый способ управления обеспечивается последовательной коммутацией фаз, при которой в каждый момент времени включена только одна обмотка. Этот способ управления называют однофазным полношаговым режимом. Схема коммутации обмоток ШД для однофазного полношагового режима показана на рис. 3.14.

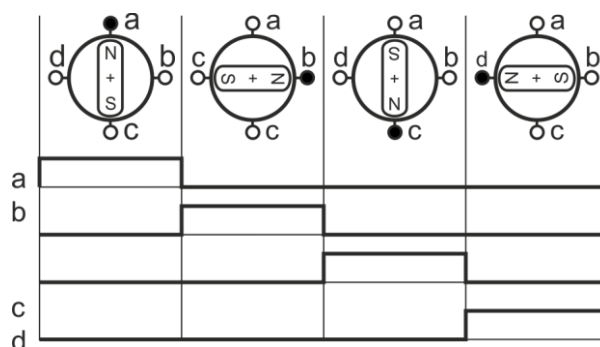


Рис. 3.14. Схема коммутации обмоток ШД для однофазного полношагового режима

Во втором способе коммутация фаз производится с перекрытием: в каждый момент времени включены две соседние обмотки двигателя и поэтому его называют двухфазным полношаговым режимом. Этот способ управления обеспечивает такой же угол шага, как и первый способ, но положение точек равновесия ротора смещено на полшага и ротор фиксируется в промежуточных позициях между полюсами статора. Кроме того, при этом способе управления обеспечивается больший момент, чем в случае одной включенной фазы. Схема коммутации обмоток ШД для реализации двухфазного полношагового режима показана на рис. 3.15.

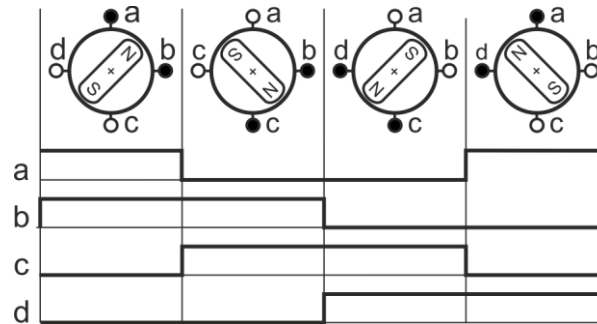


Рис. 3.15. Схема коммутации обмоток ШД для двухфазного полношагового режима

Рассмотренные способы являются простейшими способами управления ШД со всеми присущими им недостатками: малым числом шагов на оборот, повышенной шумности, колебаниями вала при перемещении от шага к шагу. От этих недостатков избавляют более сложные алгоритмы управления, к которым можно отнести следующий способ управления.

Этот способ является комбинацией первых двух и называется полушаговым режимом, при котором двигатель делает шаг, равный половине основного. Каждый второй шаг запитана лишь одна фаза двигателя, а в остальных случаях – две. В результате угловое перемещение ротора составляет половину угла шага для первых двух способов управления. Схема коммутации обмоток ШД для полушагового режима показана на рис. 3.16.

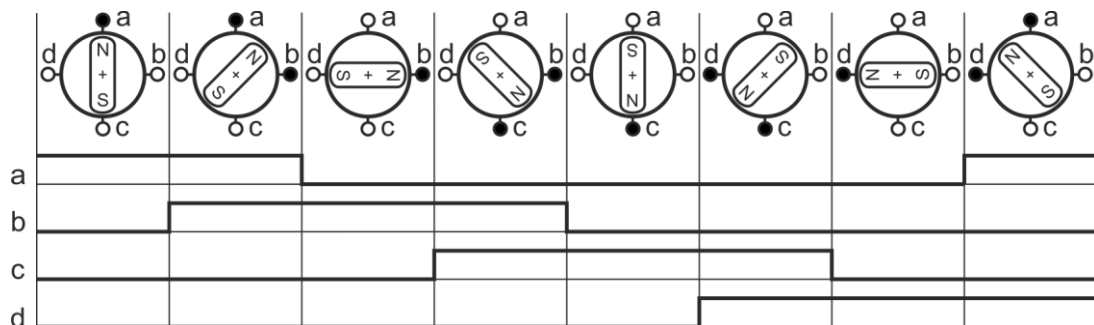


Рис. 3.16. Схема коммутации обмоток ШД для полушагового режима

Для вращения вала двигателя необходимо подавать импульсы напряжения на каждую из четырех обмоток статора в той последовательности, которая определяется используемым способом управления. Вал двигателя при этом будет на каждом шаге поворачиваться на фиксированный угол, величина которого определяется конструкцией двигателя и составляет обычно единицы градусов. Скорость вращения вала двигателя можно изменять введением пауз различной длительности между моментами переключения фаз.

Схема подключения ШД к МК показана на рис. 3.17.

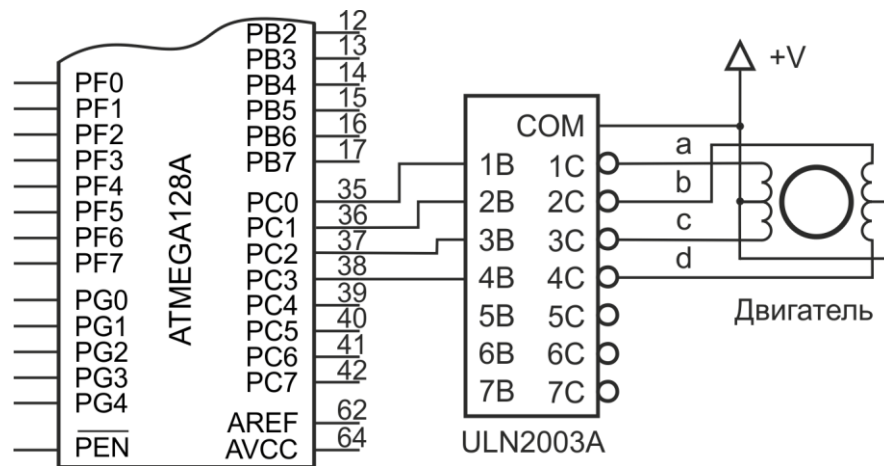


Рис. 3.17. Схема подключения шагового двигателя

Поскольку при работе ШД через его обмотки протекает ток, значительно превышающий максимально допустимый ток порта МК, то для подключения двигателя используется уже знакомая нам силовая микросхема – драйвер ULN2003A, управляемый логическими сигналами с разрядов порта.

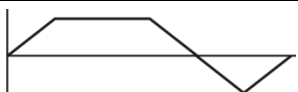
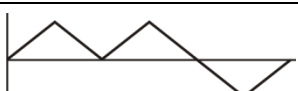
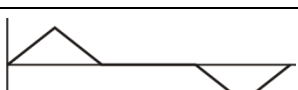
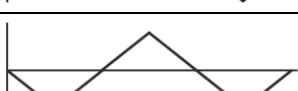
Как следует из схемы, для подачи напряжения на обмотку ШД необходимо занести в соответствующий разряд порта PORTC значение логической единицы. В этом случае на выходе драйвера появляется потенциал низкого логического уровня, и в подключенной к нему обмотке формируется передний фронт управляющего импульса. При занесении в разряд PORTC логического нуля на соответствующей обмотке формируется задний срез импульса. Длительность импульса задается введением заданной задержки между этими двумя действиями.

В качестве примера приведем фрагмент программы, реализующей выполнение одного шага в однофазном полношаговом режиме управления двигателем.

```
DDRC = 0x0F;           // младшие разряды PORTC - выход
PORTC = PORTC | (1<<0); // сформировать фронт импульса
_delay_ms(20);          // задать длительность импульса
PORTC = PORTC & ~(1<<0); // сформировать срез импульса
```

Очевидно, что скорость вращения вала двигателя можно изменять заданием длительности управляющих импульсов. При выполнении заданий следует учитывать, что предельная скорость вращения ограничена конструкцией используемого ШД и не превышает 150...200 оборотов в минуту в зависимости от режима управления.

Задание на работу №5: в соответствии с вариантом задания, представленном в таблице, составьте программу управления шаговым двигателем для реализации заданного режима управления и графика изменения скорости вращения.

Вариант задания	График изменения скорости вращения	Режим управления
1		Двухфазный полношаговый
2		Полушаговый
3		Двухфазный полношаговый
4		Полушаговый
5		Двухфазный полношаговый
6		Полушаговый
7		Двухфазный полношаговый
8		Полушаговый
9		Двухфазный полношаговый
10		Полушаговый
11		Двухфазный полношаговый
12		Полушаговый

Лабораторная работа № 6

Цель работы. Изучение внутреннего аналого-цифрового преобразователя микроконтроллера.

Аналого-цифровой преобразователь (АЦП) – устройство, преобразующее входной аналоговый сигнал в дискретный код (цифровой сигнал). АЦП имеет ряд характеристик:

Разрешение АЦП — минимальное изменение величины аналогового сигнала, которое может быть преобразовано данным АЦП — связано с его разрядностью. В случае единичного измерения без учёта шумов разрешение напрямую определяется разрядностью АЦП.

Разрядность АЦП характеризует количество дискретных значений, которые преобразователь может выдать на выходе. В двоичных АЦП измеряется в битах. Например, двоичный 8-ми разрядный АЦП, способен выдать 256 дискретных значений (0...255).

Частота дискретизации характеризует частоту выборки цифровых значений из непрерывного аналогового сигнала.

АЦП в микроконтроллере ATmega128A. ATmega128A имеет встроенный 10-и разрядный АЦП последовательного приближения. К АЦП подключается 8-и каналный аналоговый мультиплексор, позволяющий производить измерения напряжения с 8-и однополярных каналов, а также с 16-и дифференциальных каналов.

Для работы с АЦП у МК AVR есть пара регистров. Регистр мультиплексора ADMUX.

Бит	7	6	5	4	3	2	1	0	
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Н.з.	0	0	0	0	0	0	0	0	

Биты REFS_i устанавливают опорное напряжения модуля АЦП.

REFS1	REFS0	Опорное напряжение
0	0	AREF
0	1	AVCC
1	0	Зарезервировано
1	1	Внутренний источник на 2.56 В

Результат преобразования АЦП – 10 бит, поэтому результат не может быть сохранен в одном 8-битном регистре. Результат преобразования АЦП сохраняется в двух независимых регистрах (например, результат АЦП 974 = 0b1111001110, это значение занимает 10 бит, 8 бит будут сохранены в од-

ном регистре, а 2 бита будут сохранены в другом регистре). Регистры, в которых хранится результат преобразования АЦП – это ADCH и ADCL. ADCH сохраняет старшую часть результата, ADCL – младшую часть. Бит ADLAR регистра ADMUX определяет, как 10-битный результат преобразования АЦП будет распределяться в ADCH и ADCL. В случае, если ADLAR = 1, результат преобразования АЦП будет выравниваться слева, тогда ADCH будет заполнен 8 битами результата преобразования АЦП, а ADCL будет заполнен 2 младшими битами результата преобразования АЦП и эти два бита будут занимать 6-й и 7-й бит ADCL (см. рисунок ниже). Это называется левым выравниванием.

Бит	15	14	13	12	11	10	9	8	
	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADCH
Н.з.	0	0	0	0	0	0	0	0	

Бит	7	6	5	4	3	2	1	0	
	ADC1	ADC0	-	-	-	-	-	-	ADCL
Н.з.	0	0	0	0	0	0	0	0	

Биты в регистрах ADCL и ADCH хранят результат преобразования АЦП. Результат преобразования АЦП представлен как ADC0...ADC9.

Если ADLAR = 0, это означает, что результат преобразования АЦП будет выравниваться справа, тогда ADCL будет заполнен 8 битами результата преобразования АЦП, а ADCH будет заполнен 2 старшими битами результата преобразования АЦП, и эти два бита будут занимать 0-й и 1-й биты ADCH (см. рисунок ниже). Это называется правым выравниванием.

Бит	15	14	13	12	11	10	9	8	
	-	-	-	-	-	-	ADC9	ADC8	ADCH
Н.з.	0	0	0	0	0	0	0	0	

Бит	7	6	5	4	3	2	1	0	
	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	ADCL
Н.з.	0	0	0	0	0	0	0	0	

Пример: результат преобразования АЦП: 974 = 0b1111001110. В случае левого выравнивания ADCH = 0b11110011, ADCL = 0b10000000. В случае правого выравнивания ADCH = 0b00000011, ADCL = 0b11001110.

Спрашивается – зачем мне это? Ответ заключается в следующем. В случае использования левого выравнивания пользователь может прочитать

только 8 бит результата преобразования АЦП в ADCH и не тратить время на чтение ADCL, в котором хранятся только два младших бита результата преобразования АЦП (в этом случае точность АЦП снижается до 8 бит).

Биты MUX_i регистра ADMUX выбирают один из каналов АЦП (Atmega128 имеет 8 дифференциальных каналов АЦП).

$MUX4:0$	Частота АЦП	$MUX4:0$	Частота АЦП
0000	ADC0	0100	ADC4
0001	ADC1	0101	ADC5
0010	ADC2	0110	ADC6
0011	ADC3	0111	ADC7

Следующий регистр АЦП – ADCSRA.

Бит	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADFR	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Н.з.	0	0	0	0	0	0	0	0	

Включение модуля АЦП производится установкой бита ADEN. Установка бита ADSC, запускает преобразование аналогового сигнала. Если установлен бит ADFR, модуль АЦП переходит в режим циклического перезапуска. Бит ADIF – это флаг прерывания АЦП, который устанавливается после завершения преобразования АЦП и обновления регистров данных (ADCH и ADCL). Чтобы сбросить флаг, необходимо записать в него логическую единицу. Запись логической единицы в бит ADIE разрешает прерывания от АЦП. Биты ADPSi определяют частоту дискретизации модуля АЦП.

$ADPS2$	$ADPS1$	$ADPS0$	Частота тактирования АЦП
0	0	0	$F_{ADC} = F_{CPU}$
0	0	1	$F_{ADC} = F_{CPU}/2$
0	1	0	$F_{ADC} = F_{CPU}/4$
0	1	1	$F_{ADC} = F_{CPU}/8$
1	0	0	$F_{ADC} = F_{CPU}/16$
1	0	1	$F_{ADC} = F_{CPU}/32$
1	1	0	$F_{ADC} = F_{CPU}/64$
1	1	1	$F_{ADC} = F_{CPU}/128$

Ниже представлен пример функции запуска процесса преобразования АЦП и чтения 8-и битного результата.

```
uint8_t read_adc(uint8_t channel)
{
    // AVCC + левое выравнивание + выбор канала
    ADMUX = (1<<REFS0) | (1<<ADLAR) | channel;
    // Начало преобразования
```

```

ADCSRA |= (1<<ADSC) ;
// Ждем окончания преобразования АЦП
while(!(ADCSRA & (1<<ADIF))) ;
// Сброс флага завершения преобразования
ADCSRA |= (1<<ADIF) ;
// возвращаем старшие 8 бит результата
return (ADCH) ;
}

```

Задание на работу №6: написать программу управления цветом и яркостью светодиода с помощью ползункового потенциометра, подключённого к 3-му каналу АЦП, используя 10-ти битный результат преобразования.

Варианты заданий на работу

№№ Вар.	Описание исполняемого цикла: плавно изменять цвета		
	Красный цвет	Синий цвет	Зелёный цвет
1	0 ... 1023	0	0
2	0	0 ... 1023	0
3	0	0	1023 ... 0
4	0 ... 1023	0 ... 1023	0 ... 1023
5	0 ... 1023	0 ... 1023	0
6	0	1023 ... 0	0 ... 1023
7	1023 ... 0	0	1023 ... 0
8	1023 ... 0	0	0 ... 1023
9	0 ... 1023	0	0 ... 1023
10	0	1023 ... 0	1023 ... 0

Лабораторная работа № 7

Цель работы. Программирование процедур формирования периодических звуковых и световых сигналов.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.18.



Рис. 3.18. Состояние тумблеров конфигурации стенда для работы 7

Методические указания по выполнению работы

Одним из наглядных способов демонстрации работы портов является формирование на внешних выводах МК периодических сигналов с дальнейшей подачей их на световые и звуковые излучатели. В данной лабораторной установке в качестве звукового источника используется электродинамический излучатель, подключенный к разряду 4 порта PORTB, а источником света – многоцветный RGB-светодиод, соединенный с разрядами 3, 4 и 5 порта PORTE. Схема подключения к МК RGB-светодиода и звукового излучателя показана на рис. 3.19.

Как видно из схемы подключения для вывода звуковых сигналов заданной тональности необходимо сформировать программным путем периодический сигнал в звуковом диапазоне частот с последующим выводом этого сигнала на внешний звуковой излучатель через разряд PB4 порта PORTB. Транзистор Т4 используется в качестве усилителя мощности.

Для формирования одного периода такого сигнала необходимо в разряд порта записать логическую 1, выдержать паузу, длительность которой задает половину периода сигнала, затем записать логический 0 и опять выдержать паузу той же длительности.

Частота сформированного таким образом сигнала должна лежать в звуковом диапазоне частот. Для задания частоты целесообразно использовать системные функции задержки.

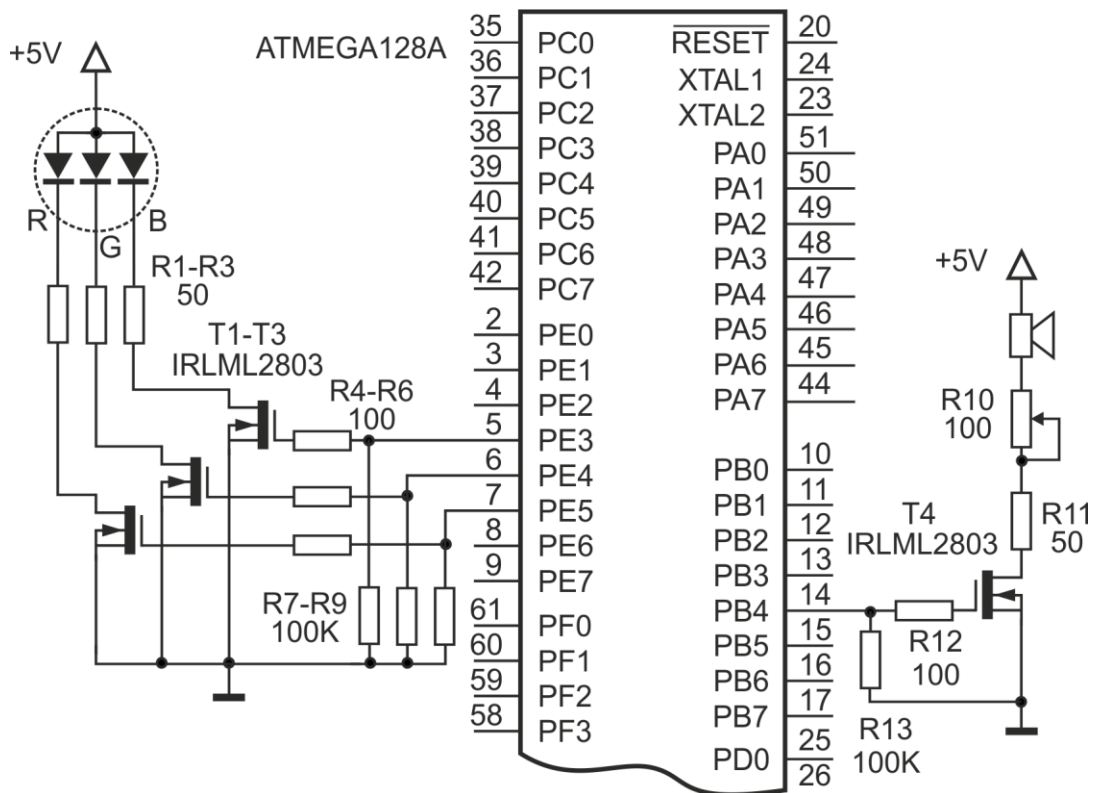


Рис. 3.19. Схема подключения RGB-светодиода и звукового излучателя

В данной лабораторной работе необходимо воспроизводить звуковые сигналы азбуки Морзе, которая представлена в таблице.

Буква	Код	Буква	Код	Буква	Код	Буква	Код
А	• —	Б	— • • •	В	• — —	Г	— — •
Д	— • •	Е	•	Ж	• • • —	З	— — • •
И	• •	Й	• — — —	К	— • —	Л	• — • •
М	— —	Н	— •	О	— — —	П	• — — •
Р	• — •	С	• • •	Т	—	У	• • —
Ф	• • — •	Х	• • • •	Ц	— • — •	Ч	— — — •
Ш	— — — —	Щ	— — • —	Ы	— • — —	Ь	— • • —
Э	• • — • •	Ю	• • — —	Я	• — • —		

Коды азбуки Морзе воспроизводятся звуковыми сигналами заданной в исходных данных варианта задания частоты, при этом точка задается сигналом длительностью 0.5 с, длительность сигнала тире – 1.5 с, интервал между символами кода равен 0.5 с, а интервал между буквами – 1 с.

При выполнении заданий в данной работе необходимо звуковые сигналы в виде кодов азбуки Морзе одновременно дублировать выводом световых сигналов заданного цвета с использованием RGB-светодиода, схема подключения которого показана на рис. 3.19. Как и в случае звукового излучателя, транзисторы Т1-Т3 используются в качестве усилителей мощности.

RGB (Red Green Blue) светодиод представляет собой трехкомпонентный световой излучатель, состоящий из трех расположенных на одном кристалле светодиодов красного, зеленого и синего цвета. Каждый из светодиодов управляется отдельно и это позволяет наряду с тремя основными цветами получать и множество других цветов путем смешения трех основных цветов различной интенсивности. Для зажигания светодиода с заданным цветом свечения необходимо подать в связанные с его выводами R, G и B разряды порта PORTE значения логической 1. Соотношения, связывающие цвет свечения излучателя с состоянием разрядов порта, показаны в таблице.

Цвет свечения	Выводы диода	Разряды PORTE		
		PE3	PE4	PE5
Красный	R	0	0	1
Зеленый	G	0	1	0
Синий	B	1	0	0
Белый	R + G + B	1	1	1
Желтый	R + G	0	1	1
Фиолетовый	R + B	1	0	1
Голубой	G + B	1	1	0

Задание на работу №7: При выполнении лабораторной работы необходимо написать программу вывода на звуковой излучатель фамилии одного из ее авторов в кодах азбуки Морзе, сопровождая звуковые сигналы выводом на RGB-светодиод световых сигналов заданного цвета. Варианты заданий с указанием частоты звукового сигнала и цвета выводимых на световой излучатель сигналов, показаны в таблице. Данную последовательность необходимо выводить в бесконечном цикле с паузой в 5 с.

Вариант	Частота звука, Гц	Цвет тире	Цвет точки	Вариант	Частота звука, Гц	Цвет тире	Цвет точки
1	100	Красный	Голубой	7	700	Красный	Фиолетовый
2	200	Зеленый	Голубой	8	800	Зеленый	Фиолетовый
3	300	Синий	Голубой	9	900	Синий	Фиолетовый
4	400	Красный	Желтый	10	1000	Красный	Белый
5	500	Зеленый	Желтый	11	1100	Зеленый	Белый
6	600	Синий	Желтый	12	1200	Синий	Белый

Лабораторная работа № 8

Цель работы. Программирование процедур вывода символьной информации на жидкокристаллический индикатор с использованием параллельного порта МК.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.20.



Рис. 3.20. Состояние тумблеров конфигурации стенда для работы 8

Методические указания по выполнению работы

Алфавитно-цифровые жидкокристаллические индикаторы (ЖКИ) представляют собой недорогое и удобное решение, позволяющее обеспечить отображение большого объема информации при хорошей различимости и низком энергопотреблении.

Управление ЖКИ требует выполнения целого ряда весьма специфических действий, таких, например, как генерация управляющих напряжений для адресации множества ячеек, формирование символов и курсора, и т. п. Для аппаратной реализации таких действий выпускаются специализированные микросхемы – контроллеры ЖКИ, которые берут на себя всю низкоуровневую часть реализации индикатора. Промышленным стандартом такой микросхемы, применяемой для построения большинства матричных знаковинтезирующих ЖКИ, является контроллер HD44780 фирмы Hitachi. Этот встроенный непосредственно в ЖКИ контроллер подключается к целевому МК через параллельный интерфейс и предоставляет программисту ряд высокоуровневых команд (например, вывод символа в указанной позиции экрана, сдвиг информации на экране или отображение мигающего курсора). Контроллер также содержит встроенный знакогенератор и экранную память, избавляя разработчика от необходимости рисовать собственные шрифты.

Основными элементами контроллера ЖКИ, с которыми приходится взаимодействовать пользователю при программном управлении, являются регистр данных DR (Data Register), регистр команд IR (Instruction Register), видеопамять DDRAM (Display Data RAM), оперативное запоминающее

устройство знакогенератора CGRAM (Character Generator RAM), счетчик адреса памяти AC (Address Counter) и флаг занятости контроллера BF (Busy Flag).

ASCII-коды отображаемых символов через регистр данных DR помещаются в видеопамять DDRAM по текущему адресу, указываемому счетчиком адреса AC. Видеопамять, имеющая общий объем 80 байт, предназначена для хранения кодов символов, отображаемых на экране ЖКИ. Видеопамять организована в виде двух буферов по 40 символов в каждом, и чаще всего длина буфера превышает число символов, выводимых в строке индикатора. В этом случае отображаются только те символы из DDRAM, которые попадают в окно индикатора, как это показано на рис. 3.21.



Рис. 3.21. Отображение символов из DDRAM в окне индикатора

В системе команд ЖКИ имеются команды, задающие смещение окна и направление этого смещения. Смещая окно индикатора относительно буфера данных, можно отображать в окне различные области буфера. Сдвиг окна индикатора относительно буфера данных для верхней и нижней строк происходит синхронно. Курсор будет виден на индикаторе только в том случае, если он попал в зону видимости DDRAM.

При внимательном рассмотрении рисунка можно заметить важную особенность. Первая строка DDRAM содержит 40 ячеек, которые адресуется от 0 до 39 (0x27), а вот 40 ячеек второй строки заданы адресами от 64 (0x40) до 103 (0x67). Значения адресов DDRAM в диапазоне 40...63 (0x28...0x3F) являются неопределенными и результаты работы с ними могут быть непредсказуемыми.

Информация, попадающая в регистр команд IR, интерпретируется контроллером ЖКИ как управляющая последовательность. Чтение регистра IR возвращает в семи младших разрядах текущее значение счетчика AC, а в старшем разряде – флаг занятости BF.

Необходимо учитывать, что большинство операций, выполняемых контроллером, занимают время около 40 мкс, а время выполнения некоторых

доходит до единиц миллисекунд. Индикатором окончания любой операции служит сброс флага занятости BF, поэтому для определения момента окончания операции программа может опрашивать флаг BF. Второй, более простой и чаще используемый способ заключается в том, что программист, зная, сколько времени требуется на обработку той или иной команды, просто выполняет в программе заведомо достаточную для выполнения команды временную задержку после каждой передачи информации в ЖКИ.

Для реализации интерфейса МК с контроллером HD44780 используется 11 информационных линий, из которых восемь (DB0...DB7) служат шиной для передачи кодов отображаемых символов или команд, а на три линии (RS, R/W, E) подаются управляющие сигналы.

При помощи сигнала на линии RS (Register Select) МК сообщает контроллеру индикатора, какой регистр – IR или DR, должен быть выбран и что будет передаваться по шине – команда или данные. Потенциал на управляющем выводе R/W задает направление передачи данных: запись в память индикатора (R/W = 0) или считывание из памяти (R/W = 1). Сигнал на линии E (Enable) является стробом, сопровождающим передачу команд и данных. Запись информации в ЖКИ происходит по переходу этого сигнала из 1 в 0.

Список всех управляющих комбинаций сигналов RS, R/W и разрядов регистра IR, а также выполняемые по ним команды приведены в таблице.

RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0	Назначение команды
0	0	0	0	0	0	0	0	0	1	Очистить дисплей и установить курсор в нулевую позицию (адрес 0)
0	0	0	0	0	0	0	0	1	*	Установить курсор в нулевую позицию (адрес 0). Установить дисплей относительно буфера DDRAM в начальную позицию.
0	0	0	0	0	0	0	1	I/D	S	Установить направление сдвига курсора влево или вправо (I/D = 0/1) при записи очередного кода в DDRAM. Разрешить (S = 1) сдвиг окна вместе со сдвигом курсора
0	0	0	0	0	0	1	D	C	B	Включить-выключить (D = 1/0) индикатор. Зажечь-погасить (C = 1/0) курсор. Сделать курсор в виде мигающего знакоместа (B = 1)
0	0	0	0	0	1	S/C	R/L	*	*	Переместить курсор (S/C = 0) или сдвинуть дисплей (S/C = 1) вправо (R/L = 1) или влево (R/L = 0)
0	0	0	0	1	DL	N	F	*	*	Установить разрядность шины данных 4 или 8 бит (DL = 0/1), количество строк – одна или две (N = 0/1), шрифт – 5 × 7 или 5 × 10 точек (F = 0/1)

0	0	0	1	AG	AG	AG	AG	AG	AG	Установка адреса CGRAM.
0	0	1	AD	AD	AD	AD	AD	AD	AD	Установка адреса DDRAM.
0	1	BF	AC	AC	AC	AC	AC	AC	AC	Чтение busy-флага BF и счетчика адреса AC
1	0	Данные								Запись данных в DDRAM или CGRAM
1	1	Данные								Чтение данных из DDRAM или CGRAM

В лабораторном стенде используется ЖКИ WH1602C-YGH фирмы Winstar, имеющий две строки по 16 символов в каждой. Схема подключения ЖКИ к МК показана на рис. 3.22.

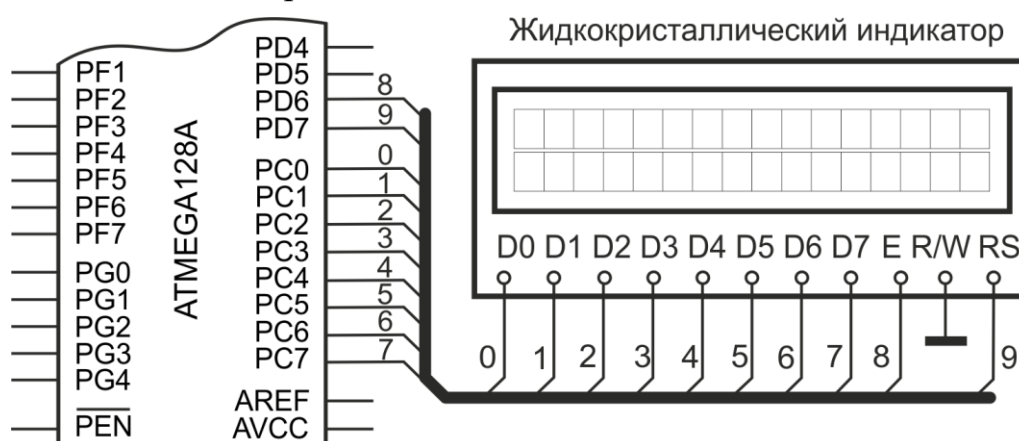


Рис. 3.22. Схема подключения ЖКИ к МК

После включения питания контроллер ЖКИ осуществляет начальную инициализацию индикатора, во время которой по умолчанию производится очистка окна и установка следующих параметров (см. таблицу команд):

- DL = 1 – разрядность шины данных равна 8;
- N = 0 – однострочный дисплей;
- F = 0 – размер знакоместа 5×7 точек;
- B = 0 – мерцание курсора отключено;
- C = 0 – курсор выключен;
- D = 0 – индикатор выключен;
- I/D = 1 – сдвиг курсора вправо;
- S = 0 – сдвиг окна отключен.

Естественно, что такой набор параметров в редком случае может быть признан подходящим (например, в нашем случае используется двустрочный дисплей). Кроме того, инициализация будет выполнена корректно, если напряжение источника питания устанавливает номинальное напряжение менее, чем за 10 мс. Поэтому перед использованием рекомендуется провести

повторную программную инициализацию индикатора с установкой необходимых параметров.

При программной инициализации индикатора для работы с 8-разрядной шиной производителем контроллера дисплея рекомендуется выполнять следующую последовательность действий.

После подачи напряжения питания необходимо выдержать паузу более 15 мс для установления номинального напряжения, но из-за различий характеристик используемых источников питания лучше задать длительность паузы в 0.1...0.2 с. Первой операцией задать RS = 0, R/W = 0 и послать команду 0x30, затем сделать задержку не менее 4.1 мс и повторить выполнение команды 0x30. На следующем шаге необходимо вновь выдержать паузу, на этот раз более 100 мкс, и в третий раз повторить команду 0x30. Эти три операции являются инициализирующими, и следом за ними выполняются команды задания режимов работы. После выполнения этих команд необходимо проверять состояние флага занятости BF, но проще выдержать паузу в 100...200 мкс и затем выдавать следующую инструкцию.

Дальнейшая последовательность команд задания режимов для индикатора лабораторного стенда такова:

1. Пауза 100...200 мкс и передача кода 0x38

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	1	DL	N	F	0	0

DL = 1 (8-битный интерфейс); N = 1 (2 строки); F = 0 (символы 5 × 7 точек).

2. Пауза 100...200 мкс и передача кода 0x0E

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	1	D	C	B

D = 1 (дисплей вкл.); C = 1 (курсор вкл.); B = 0 (мерцание выкл.).

3. Пауза 100...200 мкс и передача кода 0x06

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	0	1	I/D	S

I/D = 1 (сдвиг курсора вправо); S = 0 (нет сдвига дисплея).

Поскольку процедура инициализации ЖКИ представляет собой последовательность команд, то полезным будет предварительное создание функции записи команды в регистр команд ЖКИ

```
#define RS      7          // выбор регистра
#define E       6          // строб передачи
```

```

void LCD_cmd(uint8_t cmd) {
    //[ ]-----[ ]
    //| Назначение: запись кодов в регистр команд ЖКИ |
    //| Входные параметры: cmd - код команды |
    //[ ]-----[ ]
    DDRC = 0xFF; // все разряды PORTC на выход
    DDRD |= ((1<<E)|(1<<RS)); // разряды PORTD на выход
    PORTD &= ~(1<<RS); // выбор регистра команд RS=0
    PORTC = cmd; // записать команду в порт PORTC
    PORTD |= (1<<E); // \ сформировать на
    _delay_us(5); // | выводе E строб 1-0
    PORTD &= ~(1<<E); // / передачи команды
    _delay_ms(100); // задержка для завершения записи
}

```

С учетом этого можно реализовать процедуру инициализации ЖКИ в виде функции.

```

void LCD_init(void) {
    //[ ]-----[ ]
    //| Назначение: инициализация ЖКИ |
    //[ ]-----[ ]
    DDRC = 0xFF; // все разряды PORTC на выход
    DDRD |= ((1<<E)|(1<<RS)); // разряды PORTD на выход
    _delay_ms(100); // задержка для установления напряжения питания
    LCD_cmd(0x30); // \ вывод
    LCD_cmd(0x30); // | трех
    LCD_cmd(0x30); // / команд 0x30
    LCD_cmd(0x38); // 8 разр.шина, 2 строки, 5 x 7 точек
    LCD_cmd(0x0E); // включить ЖКИ и курсор, без мерцания
    LCD_cmd(0x06); // инкремент курсора, без сдвига экрана
    LCD_cmd(0x01); // очистить экран, курсор в начало
}

```

Используемый в лабораторном стенде индикатор русифицирован, однако коды символов кириллицы в ЖКИ не совпадают со стандартными кодами из таблицы символов ASCII и перед выводом на ЖКИ символов кириллицы необходимо производить их перекодировку. Не вдаваясь в подробности этой операции, представим функцию, реализующую процедуру такой перекодировки.

```

uint8_t TabCon[] = {0x41, 0xA0, 0x42, 0xA1, 0xE0, 0x45, 0xA3, 0xA4,
0xA5, 0xA6, 0x4B, 0xA7, 0x4D, 0x48, 0x4F, 0xA8, 0x50, 0x43, 0x54, 0xA9,
0xAA, 0x58, 0xE1, 0xAB, 0xAC, 0xE2, 0xAD, 0xAE, 0x62, 0xAF, 0xB0, 0xB1,
0x61, 0xB2, 0xB3, 0xB4, 0xE3, 0x65, 0xB6, 0xB7, 0xB8, 0xB9, 0xBA, 0xBB,
0xBC, 0xBD, 0x6F, 0xBE, 0x70, 0x63, 0xBF, 0x79, 0x5C, 0x78, 0xE5, 0xC0,
0xC1, 0xE6, 0xC2, 0xC3, 0xC4, 0xC5, 0xC6, 0xC7};

uint8_t Code(uint8_t symb) {
    //[ ]-----[ ]

```

```

//| Назначение: перекодировка символов кириллицы      |
//| Входные параметры: symb – символ ASCII             |
//| Функция возвращает код отображения символа         |
//[]-----[]
return (symb >= 192 ? TabCon[symb-192] : symb;)
}

```

Варианты заданий

Цифровая реализация хорошо апробированных на практике непрерывных законов управления в микропроцессорных автоматических системах может быть осуществлена различными методами, среди которых одним из наиболее простых является метод численного интегрирования Эйлера. При его использовании предполагается, что дифференциальное уравнение, описывающее непрерывный закон управления, задано в виде

$$dy/dt = f(y, u),$$

где f – заданная функция; y – сигнал управления; а u – входное воздействие.

Дискретизация этого уравнения по методу Эйлера приводит к следующему рекуррентному соотношению

$$y(k + 1) = y(k) + T f[y(k), u(k)],$$

где $u(k)$ и $y(k)$ – значения входного воздействия и управляющего сигнала в дискретные моменты времени $t = kT$, где T – период прерывания (шаг расчета).

В данной лабораторной работе необходимо реализовать это соотношение в виде программы для расчета управляющего сигнала $y(k)$ на пяти шагах при начальном условии $y(0) = 0$ и известных значениях $u(k)$ на каждом шаге. Для индикации полученных значений $y(k)$ необходимо на каждом шаге выводить их на индикатор, сопровождая каждый вывод трехсекундной задержкой. Формат вывода: Результат: $Y(k) = X$, где X – выводимое число, а k – номер шага. Для вариантов с четными номерами вывод производится в первой строке ЖКИ, а с нечетными – во второй. Варианты заданий представлены в таблицы.

№ варианта	Функция	Т	Значения $u(k)$				
			0	1	2	3	4
1	$f = -2y + u/2 - 3$	1	8	12	16	20	24
2	$f = -4y + 2u - 5$	2	3	7	11	15	19
3	$f = 3y + u/2 - 5$	3	12	-6	-24	-42	-60
4	$f = 4y + u/2 - 11$	1	24	16	8	0	-8

Продолжение Таблицы

5	$f = y + 2u + 3$	2	-1	-2	-3	-4	-5
6	$f = 2y + u/3 + 4$	3	-9	-27	-45	-63	-81
7	$f = -5y + u/4 - 1$	1	8	28	48	68	88
8	$f = 4y + 2u + 7$	2	-3	-7	-11	-15	-19
9	$f = 7y + 3u - 2$	3	1	-6	-13	-20	-27
10	$f = 2y - u/8 + 3$	3	12	48	84	120	156
11	$f = -3y + 3u - 5$	1	2	3	4	5	6
12	$f = y + u/4 - 1$	2	8	0	-8	-16	-24

Лабораторная работа № 9

Цель работы. Программирование процедур ввода информации через параллельный порт МК с использованием клавиатуры.

Перед выполнением работы установите тумблеры конфигурации стенда в состояние, показанное на рис. 3.23.



Рис. 3.23. Состояние тумблеров конфигурации стенда для работы 9

Методические указания по выполнению работы

Одним из широко распространенных в системах управления и контроля устройств, предназначенных для ввода информации в МК, является матричная клавиатура. В стенде используется пленочная клавиатура с организацией в виде матрицы 4×3 . Схема подключения клавиатуры к МК показана на рис. 3.24.

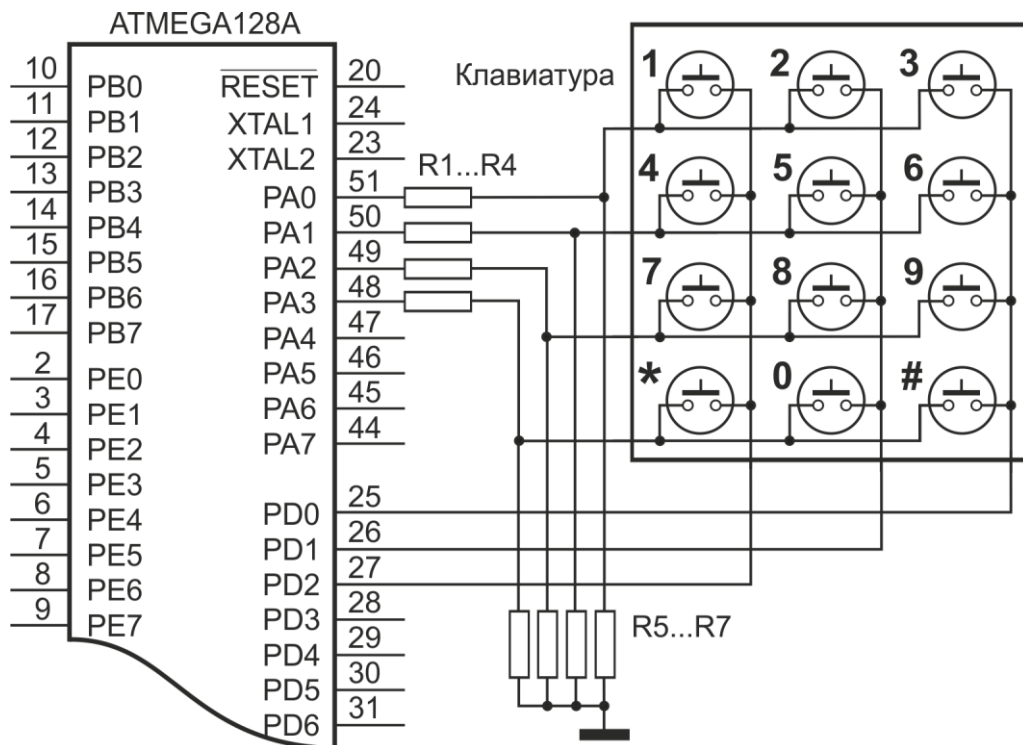


Рис. 3.24. Схема подключения клавиатуры к МК

Один вывод каждой клавиши соединяется с вертикальной линией сканирования, а второй вывод – с горизонтальной линией возврата. Обнаружение нажатой клавиши и ее последующая идентификация основаны на использовании

процедуры сканирования, которая сводится к поочередной установке каждой из линий сканирования в состояние логической единицы (при одновременном обнулении остальных) и опросу состояния линий возврата.

В соответствии с этим в разряды PD0, PD1 и PD2 порта PORTD, настроенные на выход (линии сканирования), циклически выдаются коды сканирования, содержащие логическую 1 только в одном разряде и логические 0 в остальных.

В разряды PA0...PA3 порта PORTA, настроенные на вход без внутренних подтягивающих резисторов (линии возврата), после каждого изменения кода сканирования считывается состояние линий строк клавиатуры.

Если клавиши не нажаты, то при любом состоянии линий сканирования со всех разрядов линий возврата считываются логические 0, заданные внешними подтягивающими резисторами R5...R7. При нажатии любой клавиши линия сканирования замыкается с линией возврата и, когда в линию сканирования с нажатой клавишей поступит логическая 1, то в соответствующей линии возврата тоже считается логическая 1.

Зная номер линии сканирования с заданной логической 1 (столбец), и номер линии возврата (строка), где появилась логическая 1, мы можем легко определить нажатую клавишу.

Есть, однако, одна особенность, которую необходимо учитывать при разработке программы. Во время работы МК с датчиками, имеющими механические контакты (кнопки, клавиши, клавиатуры и т. д.), возникает явление, называемое дребезгом. Оно заключается в том, что в момент соприкосновения или разъединения контактов сопротивление между ними хаотически меняется в течение некоторого времени, зависящего от материала, конфигурации и состояния контактов, воздействующей на контакты силы и ряда других факторов. Во время дребезга сигнал с контакта может быть прочитан как случайная последовательность нулей и единиц. Подавить это нежелательное явление можно аппаратными средствами, но чаще это делается программным путем.

Наиболее простым способом подавления эффекта дребезга контактов является введение временной задержки. Он основан на том, что программа, обнаружив первое замыкание контакта, запрещает опрос состояния этого контакта на время, заведомо большее длительности времени дребезга. Время задержки лежит в пределах 10...30 мс и подбирается экспериментально для каждого типа контактов. Повторный опрос состояния клавиши после задержки дает истинное значение этого состояния.

Окончательно алгоритм опроса состояния клавиатуры необходимо представить в виде отдельной функции. В начале этой функции разряды PD0, PD1 и PD2 порта PORTD настраиваются на выход, а разряды PA0...PA3 порта PORTA – на вход без подключения внутренних подтягивающих резисторов. Затем в ней реализуется бесконечный цикл, в котором выполняется следующая последовательность действий:

- 1) задать цикл формирования и записи в разряды PD0, PD1 и PD2 порта PORTD кодов сканирования с «бегающей» единицей;
- 2) после записи каждого кода сканирования считать и проверить состояние разрядов PA0...PA3 порта PORTA;
- 3) если во всех разрядах считано состояние логического 0, то закончить очередной цикл сканирования и перейти к п. 7;
- 4) если хотя бы в одном разряде считано состояние логической 1, то ввести паузу для подавления дребезга;
- 5) произвести повторное считывание разрядов PA0...PA3 порта PORTA, дающее истинное состояние нажатой клавиши;
- 6) по номерам столбца и строки, содержащих логическую 1, определить, какая клавиша была нажата и код нажатой клавиши вывести по команде возврата из функции;
- 7) конец цикла задания кода сканирования.

Варианты заданий

В данной работе используются навыки по программированию вывода информации на ЖКИ индикатор, полученные при выполнении предыдущей лабораторной работы. Во всех вариантах ввод исходных данных и вывод результата индицируются на ЖКИ индикаторе.

1. Разработать калькулятор, выполняющий операцию сложения двух целых десятичных чисел. С клавиатуры вводятся слагаемые, операция сложения реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

2. Разработать калькулятор, выполняющий операцию поразрядного логического И над двумя десятичными числами. С клавиатуры вводится первое число, затем после нажатия клавиши * – второе число, а результат выводится после нажатия клавиши #.

3. Разработать калькулятор, выполняющий операцию вычитания двух целых десятичных чисел. С клавиатуры вводятся числа, операция вычитания

реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

4. Разработать калькулятор, выполняющий операцию поразрядного логического ИЛИ над двумя десятичными числами. С клавиатуры вводится первое число, затем после нажатия клавиши * – второе число, а результат выводится после нажатия клавиши #.

5. Разработать калькулятор, выполняющий операцию умножения двух целых десятичных чисел. С клавиатуры вводится числа, операция умножения реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

6. Разработать калькулятор, выполняющий поразрядную операцию ИСКЛЮЧАЮЩЕЕ ИЛИ над двумя десятичными числами. С клавиатуры вводится первое число, затем после нажатия клавиши * – второе число, а результат выводится после нажатия клавиши #.

7. Разработать калькулятор, выполняющий операцию целочисленного деления двух целых десятичных чисел. С клавиатуры вводятся числа, операция деления реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

8. Разработать калькулятор, выполняющий операцию сдвига влево целого десятичного числа. С клавиатуры вводится число, после нажатия клавиши * – число разрядов, на которое производится сдвиг, а результат выводится после нажатия клавиши #.

9. Разработать калькулятор, выполняющий операцию возведения в степень целого десятичного числа. С клавиатуры вводится число, после нажатия клавиши * – показатель степени, а результат выводится после нажатия клавиши #.

10. Разработать калькулятор, выполняющий операцию сдвига вправо целого десятичного числа. С клавиатуры вводится число, после нажатия клавиши * – число разрядов, на которое производится сдвиг, а результат выводится после нажатия клавиши #.

11. Разработать калькулятор, выполняющий операцию остатка от целочисленного деления двух целых десятичных чисел. С клавиатуры вводятся числа, операция вычисления остатка реализуется нажатием клавиши *, а результат выводится после нажатия клавиши #.

12. Разработать калькулятор, выполняющий операцию поразрядной инверсии целых десятичных чисел. С клавиатуры вводятся число, операция инверсии реализуется нажатием клавиши #.

Битовые операции для целочисленных операндов

В языке Си имеются шесть операторов (&, |, ^, ~, >>, <<), которые можно использовать в программе для манипулирования битами в целых числах.

Оператор & (поразрядное «И») выполняет операцию логического умножения отдельных одноименных разрядов двух операндов.

Оператор | (поразрядное «ИЛИ») реализует операцию логического сложения отдельных одноименных битов двух операндов.

Операторы & и |, которые работают с операндами побитно, следует отличать от логических операторов && и ||, которые оценивают каждый операнд целиком, равен он нулю или нет. Например, если переменная Var1 = 1, а переменная Var2 = 4, то результат побитовой операции (Var1 & Var2) будет равен нулю, а логической (Var1 && Var2) будет равен единице.

Оператор ^ (поразрядное исключающее «ИЛИ») реализует сложение по модулю 2 для одноименных разрядов.

Оператор инверсии ~ (поразрядное отрицание) инвертирует все биты переменной.

Операции сдвига перемещают разряды операнда влево (<<) или вправо (>>) на указанное число позиций.

Часто операции сдвига используют для быстрого умножения или деления операнда на степень двойки (2, 4, 8, ...). Каждый сдвиг влево на один разряд приводит к умножению операнда на два, а каждый сдвиг вправо – к делению на два.

Рассмотрим более подробно битовые операции и их применение при программировании МК.

Оператор сдвига влево <<

Оператор сдвига влево << выполняет поразрядный сдвиг числа влево. Его синтаксис:

Var1 = Var2 << Num;

где Var1 и Var2 – целочисленные переменные, а Num – число разрядов для сдвига.

Двоичное значение переменной Var2 сдвигается влево на указанное количество разрядов Num с заполнением освобождающихся справа разрядов нулями. При выходе за старший разряд сдвигаемые биты теряются.

Если Var1 и Var2 – одна переменная, то можно использовать укороченную форму записи

Var <<= Num;

Пример:

```
uint8_t n = 0x01; // n = 0b00000001
n = n << 4;           // n = 0b00010000
uint8_t m = 0x03; // m = 0b00000011
m <<= 2;           // m = 0b00001100
```

Помимо переменных можно сдвигать числовые константы, например:

```
uint8_t n;
n = 1 << 5;
```

В этом примере единица, представленная своим двоичным значением 0b00000001, сдвигается влево на пять разрядов, и переменная n принимает двоичное значение 0b00100000.

Оператор сдвига вправо >>

Оператор сдвига вправо >> выполняет поразрядный сдвиг числа вправо. Его синтаксис:

Var1 = Var2 >> Num;

Значение переменной Var2 сдвигается вправо на указанное количество разрядов Num с заполнением освободившихся разрядов слева нулями. Следует отметить, что данное утверждение верно, если переменная Var2 беззнакового типа, или если переменная знаковая и ее текущее значение положительное. Если переменная знаковая и ее значение отрицательное, то старшие разряды заполняются единицами. При выходе за нулевой разряд переменной сдвигаемые биты теряются.

Пример:

```
uint8_t n = 0x80; // n = 0b10000000
n = n >> 4;           // n = 0b00001000
uint8_t m = 0xC0; // m = 0b11000000
m >>= 2;           // m = 0b00110000
```

Оператор инверсии ~

Оператор ~ инвертирует каждый разряд операнда, т.е. заменяет 1 на 0 и наоборот. Этот оператор – унарный и применяется только к одному операнду.

Var1 = ~Var2;

Пример:

```
uint8_t n = 0xAA; // n = 0b10101010
n = ~n;           // n = 0b01010101
```

Оператор логического умножения &

Оператор & производит над операндами логическую операцию поразрядного "И". Это значит, что разряд результата будет равен 1 только в том случае, когда соответствующие разряды обоих операндов содержат 1. В противном случае он будет равен 0.

Var1 = Var2 & Var3;

Пример:

```
uint8_t a, b, c;
a = 0x55;           // a = 0b01010101
b = 0x01;           // b = 0b00000001
c = a & b;           // c = 0b00000001
```

Оператор & часто используется для принудительной очистки заданных разрядов одного из операндов, для чего в качестве второго операнда задается так называемая маска, в которой разряды, требующие очистки, установлены в 0.

Пример:

```
uint8_t a, mask;
a = 0x75;           // a = 0b01110101
mask = 0x0F;        // mask = 0b00001111
a = a & mask;        // a = 0b00000101
```

Оператор логического сложения |

Оператор | производит над операндами логическую операцию поразрядного "ИЛИ". Это значит, что разряд результата будет равен 1, если 1 содержится в соответствующем разряде хотя бы одного операнда.

Var1 = Var2 | Var3;

Пример:

```
uint8_t a, b, c;
a = 0x55;           // a = 0b01010101
b = 0x81;           // b = 0b10000001
c = a | b;           // c = 0b11010101
```

Оператор | применяется для принудительной установки заданных разрядов одного из операндов, при этом во втором операнде задается маска, в которой разряды, подлежащие установке в единицу, равны 1.

Пример:

```
uint8_t a, mask;  
a = 0x05;           // a = 0b00000101  
mask = 0xF0;        // mask = 0b11110000  
a = a | mask;        // a = 0b11110101
```

Оператор сложения по модулю 2 ^

Оператор ^ производит над операндами логическую операцию исключающего "ИЛИ", при которой в каждый разряд результата записывается 1, если соответствующие разряды операндов различаются, и 0, если они совпадают.

Var1 = Var2 ^ Var3;

Пример:

```
uint8_t a, b, c;  
a = 0x55;           // a = 0b01010101  
b = 0x03;           // b = 0b00000011  
c = a ^ b;          // c = 0b01010110
```

Оператор ^ используется для инверсии заданных битов одного из операндов и для этого во втором операнде задается маска, в которой биты, подлежащие инверсии, установлены в 1.

Пример:

```
uint8_t a, mask;  
a = 0x55;           // a = 0b01010101  
mask = 0xF0;        // mask = 0b11110000  
a = a ^ mask;        // a = 0b10100101
```

Работа с отдельными битами

В МК для задания режимов работы периферийных устройств и управления ими используются специальные регистры, каждый разряд которых привязан к конкретному устройству и задание соответствующих функций определяется содержимым этого разряда. В подобных случаях для манипуляций с битами применяют рассмотренные выше побитовые операции, при этом в качестве второго операнда чаще всего используется переменная, имеющая 1 на месте значимых бит. Остальные незначимые биты должны быть невидимыми (замаскированными) для операций установкой 0. Поэтому указанную переменную для операций с битами часто называют маской.

Рассмотрим некоторые приемы манипуляций с битами, которые часто используются при программировании МК.

Запись единицы в заданный разряд с обнулением остальных разрядов

Самым простым и естественным способом записи одной или нескольких единиц с обнулением остальных разрядов является обычное присвоение переменной значения с единицами в требуемых разрядах и нулями в остальных, например:

```
uint8_t a, b;  
a = 0x10;           // a = 0b00010000  
b = 0x30;           // b = 0b00110000
```

Альтернативный и часто используемый программистами вариант основан на применении оператора сдвига влево:

```
Var = 1 << Num;
```

Пример:

```
uint8_t a;  
a = 1 << 4;          // a = 0b00010000
```

Его легко расширить для записи в разряды переменной нескольких единиц с обнулением остальных разрядов:

```
Var = (1<<Num1) | (1<<Num2) | ... | (1<<NumN) ;
```

Пример:

```
uint8_t a;  
a = (1 << 5) | (1 << 4); // a = 0b00110000
```

Запись единицы в заданный разряд без изменения остальных разрядов

Для записи единицы в некоторый разряд без изменения остальных разрядов можно воспользоваться маской, содержащей единицы в требуемых разрядах и выполнить над ней и исходной переменной операцию поразрядного логического "ИЛИ", например:

```
uint8_t a, mask;  
mask = 0x30;           // mask = 0b00110000  
a = 0x03;              // a = 0b00000011  
a = a | mask;          // a = 0b00110011
```

Альтернативный вариант – применение оператора сдвига

```
Var = Var | (1 << Num) ;
```

или в укороченной форме записи

```
Var |= (1 << Num) ;
```

Модификация этого способа для записи нескольких логических единиц без изменения остальных разрядов выглядит так:

```
Var |= (1<<Num1) | (1<<Num2) | ... | (1<<NumN) ;
```

Пример:

```
uint8_t a;  
a = 0x03;           // a = 0b00000011  
a |= (1 << 4) | (1 << 5); // a = 0b00110011
```

Запись нуля в заданный разряд без изменения остальных разрядов

Для записи нуля в некоторый разряд без изменения остальных разрядов можно воспользоваться оператором:

```
Var = Var & ~(1 << Num) ;
```

Эта запись равнозначна следующей:

```
Var &= ~(1 << Num) ;
```

Запись нескольких нулей без изменения остальных разрядов:

```
Var &= ~( (1<<Num1) | (1<<Num2) | ... | (1<<NumN) ) ;
```

Пример:

```
uint8_t a;  
a = 0x87;           // a = 0b10000111  
a &= ~( (1 << 1) | (1 << 2) ); // a = 0b10000001
```

Можно также применить к переменной операцию поразрядного "И" с маской, в которой в позициях обнуляемых разрядов должны быть нули.

Пример:

```
uint8_t a, mask;  
mask = 0xF9;           // mask = 0b11111001  
a = 0x87;           // a = 0b10000111  
a = a & mask;       // a = 0b10000001
```

Инвертирование отдельных битов

Для инвертирования содержимого в заданном разряде целочисленной переменной используется операция поразрядного исключающего «ИЛИ» с помощью оператора:

```
Var ^= (1 << Num) ;
```

Инверсия нескольких разрядов без изменения остальных:

```
Var ^= (1<<Num1) | (1<<Num2) | ... | (1<<NumN) ;
```

Пример:

```
uint8_t a, b;  
a = 0x87;           // a = 0b10000111  
a ^= (1 << 7);      // a = 0b00000111  
b = 0x87;           // b = 0b10000111  
b ^= (1 << 7) | (1 << 1); // b = 0b00000101
```

Проверка значения бита в переменной

Для проверки значения любого бита в переменной необходимо обнулить все биты, кроме проверяемого, а потом сравнить полученное значение с нулем

```
if ((Var & (1<<Num)) != 0 ) {  
    // блок выполнится, если бит Num переменной Var установлен  
}  
if ((Var & (1<<Num)) == 0 ) {  
    // блок выполнится, если бит Num переменной Var сброшен  
}
```

Обмен частей переменной

Одной из типовых операций преобразования данных является обмен тетрад в байте, когда четыре младших бита помещаются на место старших, а четыре старших бита – на место младших.

Обменять тетрады любой беззнаковой байтовой переменной можно при помощи двух операций сдвига:

```
Var = (Var << 4) | (Var >> 4);
```

В первой операции сдвига биты младшей тетрады перемещаются в старшую, а младшая тетрада при сдвиге обнуляется. Во второй операции сдвига старшая тетрада перемещается на место младшей. Затем операция поразрядного «ИЛИ» объединяет байты со сдвинутыми тетрадами.

Использование макросов

Директива `#define` служит для замены часто использующихся в программе констант, операторов или выражений некоторыми идентификаторами. Идентификаторы, заменяющие текстовые или числовые константы, называют именованными константами, а идентификаторы, заменяющие фрагменты программ, называют макроопределениями. Директива `#define` имеет две синтаксические формы:

```
#define идентификатор текст  
#define идентификатор(список параметров) текст
```

Эта директива заменяет все последующие вхождения идентификатора в программе на текст.

Пример:

```
#define MIN 10  
#define MAX (MIN+20)
```

Эти директивы заменят в тексте программы каждое слово MIN на число 10, а каждое слово MAX на выражение (10+20) вместе с окружающими его скобками. Скобки, содержащиеся в макроопределении, позволяют избежать ошибок, связанных с порядком вычисления операций. Например, при отсутствии скобок выражение $a = MAX * 5$ будет преобразовано в выражение $a = 10 + 20 * 7$, а не в выражение $a = (10 + 20) * 7$, как это получается при наличии скобок.

При разработке компиляторов данная директива чрезвычайно широко используется для привязки адресов аппаратных ресурсов МК к их символическим именам, заданным в технической документации на МК. Если заглянуть, например, в любой заголовочный файл конкретного МК, то можно увидеть, что большая его часть состоит из директив `#define`. Использование заданных таким образом символических имен портов, регистров и даже отдельных разрядов, позволяет резко упростить написание программ.

При создании прикладных программ применение директивы также очень удобно и позволяет разрабатывать программы, в минимальной степени зависящие от использования конкретных аппаратных ресурсов МК.

Например, если в проекте при подключении клавиатуры использовался порт PORTA, то можно задать привязку этого подключения так

```
#define P_KEY PORTA
```

Тогда в дальнейшем в программе можно использовать символическое имя P_KEY. Если по каким-то причинам возникнет необходимость изменить порт подключения клавиатуры, то сделать это в программе нужно будет только в одном месте – в директиве `#define`, а не искать места его использования по всей программе.

В директиве можно использовать и выражения, например

```
#define MASK ((1 << 1) | (1 << 2))
```

Макросы создаются с использованием второй синтаксической формы директивы `#define`. При идентификаторе макроса имеется список формальных параметров, заключенных в скобки и разделенных запятыми. Формальные параметры в тексте макроопределения отмечают позиции, в которые должны быть подставлены фактические аргументы макровывода.

При вызове макроса вслед за идентификатором записывается список фактических аргументов, количество которых должно совпадать с количеством формальных параметров в макроопределении.

Пример:

```
#define MAX(x,y) ((x)>(y))?(x):(y)
```

Эта директива заменит фрагмент

```
a = MAX(i,f);
```

на фрагмент

```
a = ((i)>(f))?(i):(f);
```

Как и в предыдущем примере, круглые скобки, в которые заключены формальные параметры макроопределения, позволяют избежать ошибок связанных с неправильным порядком выполнения операций, если фактические аргументы являются выражениями.

При программировании МК наиболее часто применяются макросы, основанные на использовании рассмотренных выше битовых операций. Рассмотрим самые распространенные из них.

Макрос установки заданного бита в переменной или регистре:

```
#define BitSet(var, bit) ((var) |= (1<<(bit)))
```

Пример:

```
BitSet(PORTA,5); // установить 5-й бит PORTA
```

Макрос очистки заданного бита в переменной или регистре:

```
#define BitClr(var, bit) ((var) &= ~(1<<(bit)))
```

Пример:

```
BitClr(PORTC,4); // очистить 4-й бит PORTC
```

Макрос инверсии заданного бита в переменной или регистре:

```
#define BitInv(var, bit) ((var) ^= (1<<(bit)))
```

Пример:

```
BitInv(PORTB,1); // инвертировать 1-й бит PORTB
```

Макрос возвращает 1, если заданный бит в переменной или регистре равен 1:

```
#define BitIsSet(var, bit) ((var & (1<<bit)) != 0)
```

Пример:

```
if(BitIsSet(PORTB,1)){Блок оператора if};
```

Макрос возвращает 1, если заданный бит в переменной или регистре равен 0:

```
#define BitIsClr(var, bit)    ((var & (1<<bit)) == 0)
```

Пример:

```
if(BitIsClr(PORTB,1)){Блок оператора if};
```

Данные макросы можно поместить в заголовочный файл (например, macros.h) и использовать его в своих программах. При появлении новых макросов можно легко расширять библиотеку, помещая их в этот файл.